



GE VERNOVA

ADMS

16.25.1

Internal Notifications Installation Guide

Copyright

Copyright 2026, GE Vernova and/or its affiliates ("GE Vernova"). All Rights Reserved.

GE and the GE Monogram are trademarks of General Electric Company used under trademark license.

This document is the confidential and proprietary information of GE Vernova and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE Vernova.

Change History

Date	Change Description
December 08, 2023	Updated document format and styling.
April 25, 2024	This document has been updated with 'Deployment based on Helm chart'.
Oct 14, 2024	Updated section Importing Certificates to the ADMS Hosts .

Contents

1. Introduction	5
2. Installation Overview	7
2.1. Hardware and Operating System Prerequisites	7
2.2. Software Prerequisites	8
2.2.1. Server	8
2.2.2. Deployment Server	8
2.2.3. Clients	8
2.3. Software Delivery Media	8
3. Setting Up a Deployment Server (Air Gapped)	10
3.1. Installing the Nexus Repository	10
3.2. Configuring Nexus for Kubernetes Nodes	11
3.3. Uploading Artifacts to Nexus	13
4. Setting Up Internal Notifications	15
4.1. Configuring Foundation Base Components	15
4.2. Configuring Internal Notifications Components	15
4.2.1. Preparing Local Setup Folders	15
4.2.2. Configuring Message Formats	16
4.2.3. Configuring the Core Notification Consumer	20
4.2.4. Generating Certificates	21
4.2.5. Internal Notifications: Required Permissions and Roles	23
4.2.5.1. Roles	23
4.2.5.2. Permissions	23
4.2.5.3. Roles Mappings	23
4.2.5.4. Usage Notes	24
4.2.5.5. Default Role Assignments in Foundation Deployment	24
4.2.6. Configuring the YAML Files	25
4.2.6.1. Configuring the Internal Notifications YAML File	26
4.2.6.2. Internal Notifications Consumer	26
4.2.6.3. Internal Notifications Service	26
4.2.6.4. Internal Notifications UI	27
4.2.6.5. Global Parameters	27
4.2.6.6. Creating secrets for Internal Notifications	29
4.3. Database Credentials	29
4.4. SMTP Settings	30
4.5. UAA Client Secret	30
4.6. Encryption key Secret	31
4.6.1. Configuring the hosts.etc File	37
4.6.2. Setting Up Firewall Rules	38
4.6.3. Importing Certificates to the ADMS Hosts	38
4.7. Deploying Internal Notifications to Kubernetes	43

4.8. Configuring the Core Notification Server	46
4.8.1. Configuring Credentials to Access the Kubernetes Cluster	47
4.9. Configuring the Call Server	47
5. Running Internal Notifications	49
6. Maintenance and Troubleshooting	51
6.1. Verifying the Setup	51
6.2. Upgrading to a Newer Version	52
6.3. Enable Debug Logging	52
6.3.1. Appsettings.json	54
6.3.2. Statefulset config	54
6.4. Enabling Additional Logging	55
6.5. Configuring Automatic Database Clean-Up	55
6.6. Deleting Records in the Outage Table	56
6.7. Troubleshooting	57
6.7.1. Notifications Contain Duplicate Service Center Data	57
6.7.2. Area Filter is Blank	57
6.7.3. Customer Search is not Working	57
6.7.4. Subscribers do not Receive Email Notifications	57

1. Introduction

The Internal Notifications application informs internal staff about certain OMS events via email or SMS using a subscription model. Internal stakeholders can register to receive different types of notifications from the system.

The supported notification types include:

- **Affected Customers:** Notifications about specific customers involved in an outage.
- **Customer Count:** Notifications for outages where the number of affected customers exceeds a specified threshold.
- **Device Operations:** Notifications for outages that involve specific device types, such as breakers or reclosers.

The Internal Notifications solution uses the security component of GE's common Kubernetes-based platform (hereinafter referred to as "Foundation"). The Foundation Base and Internal Notifications deployment, scaling, and management is implemented using Kubernetes.

Clients connect to the Internal Notifications application using a web browser (Google Chrome is recommended) with an enabled JavaScript engine.

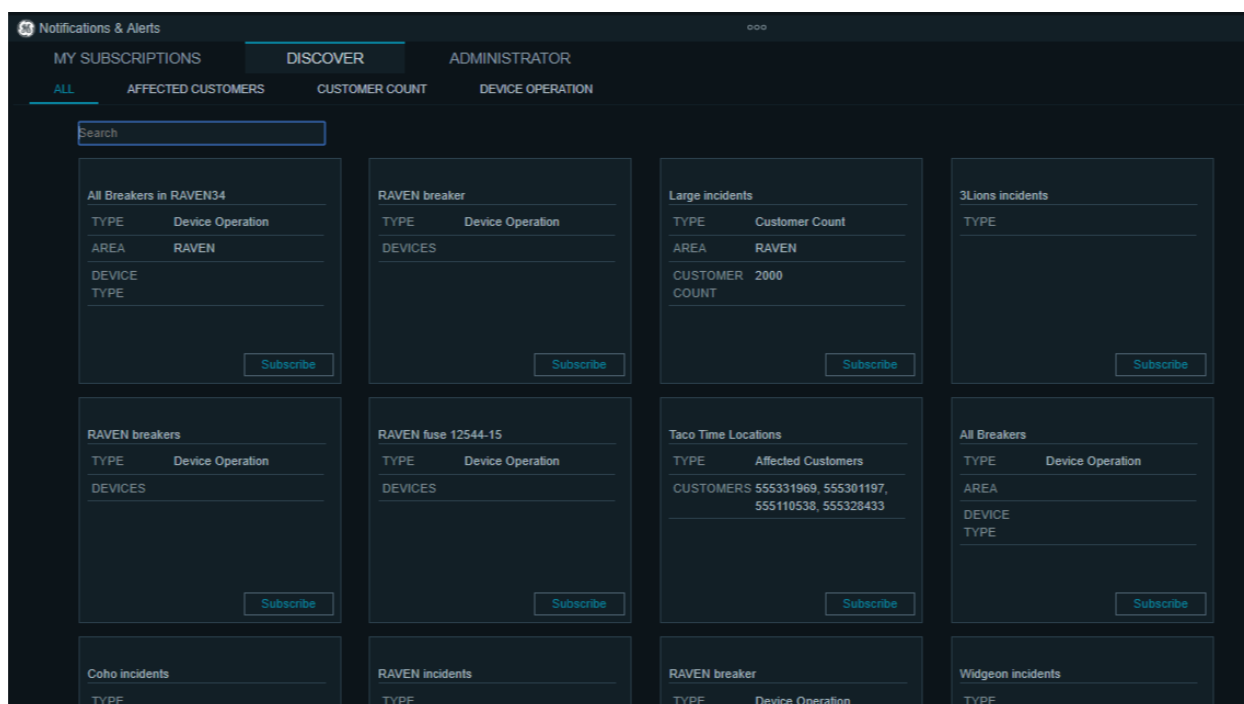


Figure 1. Internal Notifications Application

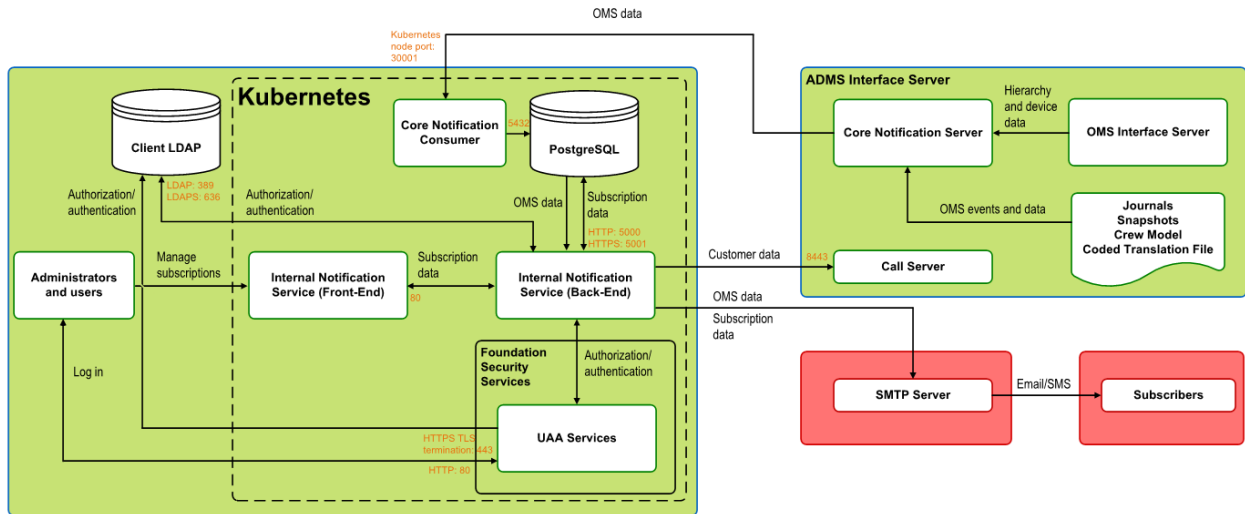


Figure 2. Internal Notifications Process

The following figure shows the Internal Notifications database in PostgreSQL.

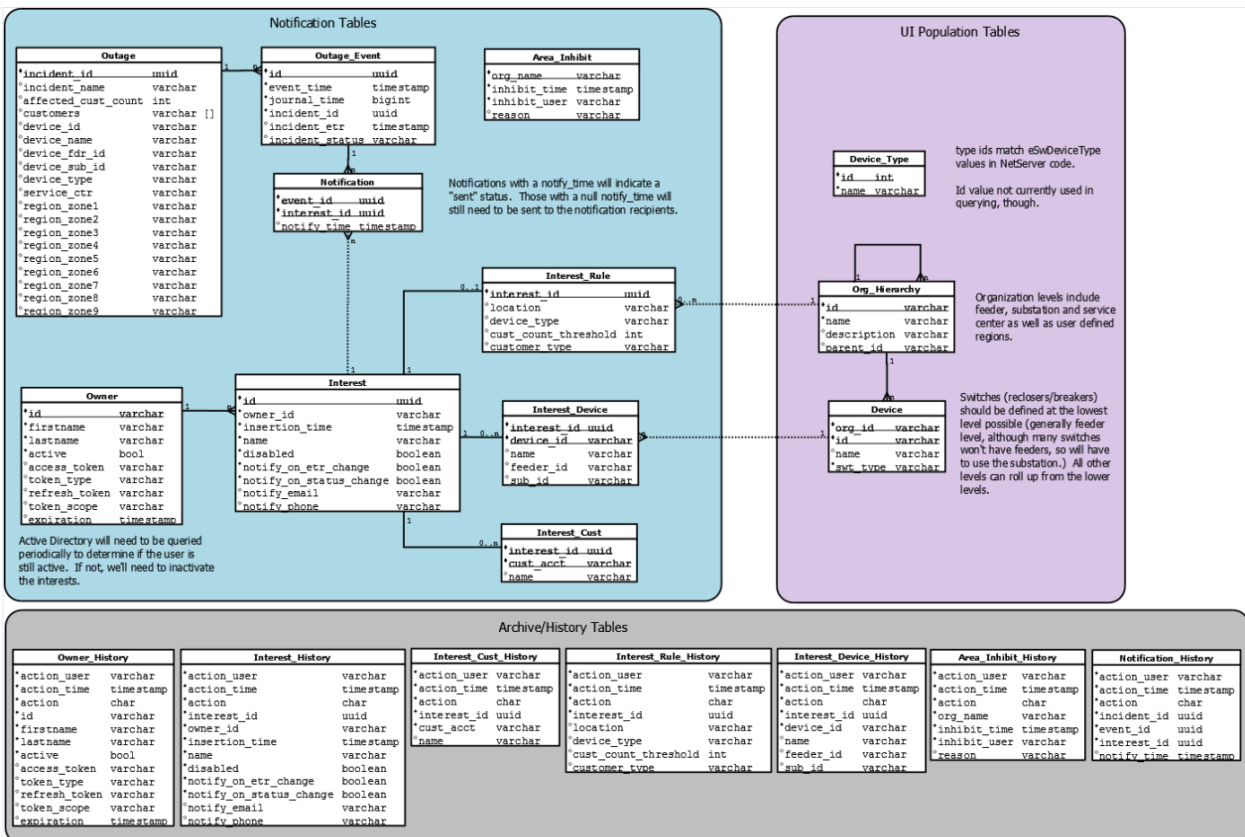


Figure 3. Internal Notifications Database

2. Installation Overview

Internal Notifications components (containers) are orchestrated by Kubernetes. Therefore, you must first create a Kubernetes environment before deploying Internal Notifications. The application deployment is certified for a "vanilla" (open source) Kubernetes cluster.

Internal Notifications deployment is based Helm chart and ArgoCd, which deploys Foundation Base and Internal Notifications to the Kubernetes cluster.

Helm helps you define, install, and upgrade Kubernetes applications and manage Helm charts.

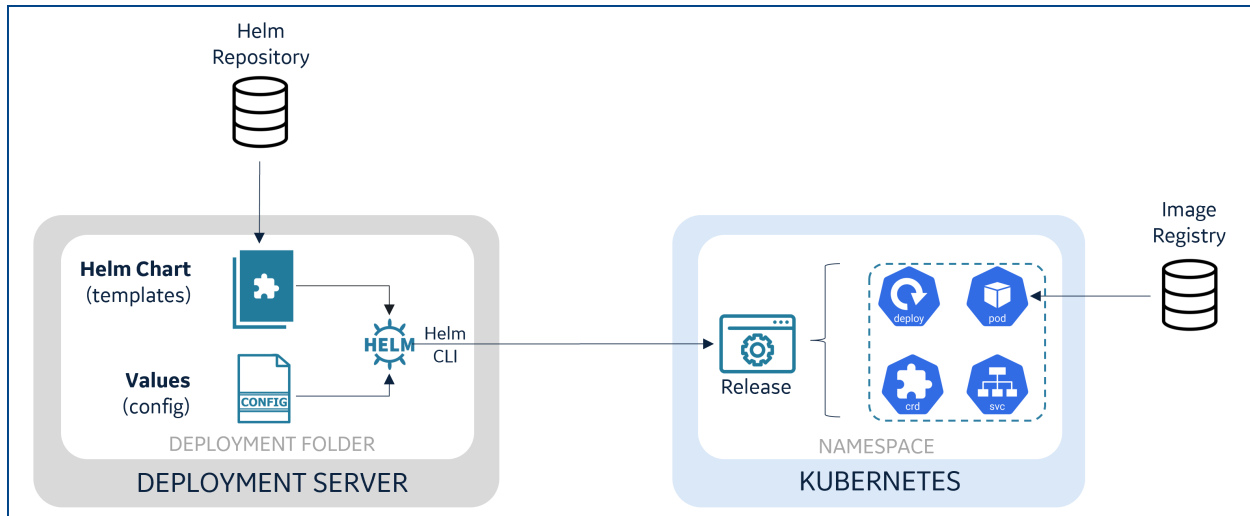


Figure 4. Deployment Overview

For more information about deploying the application, see [Setting Up a Deployment Server \(Air Gapped\)](#) and [Setting Up Internal Notifications](#).

2.1. Hardware and Operating System Prerequisites

Hardware requirements for Internal Notifications are the following:

- **Kubernetes Server:** A high availability topology-capable (HA) cluster setup is highly recommended. Use at least one node with the following properties:
 - **Memory:** 8 GB per node
 - **CPU:** 2 cores
 - **Storage:** At least 20 GB per node
 - **Operating System:** Red Hat Enterprise Linux 7.5 or later.
- **Deployment Server:** A server specifically for deployment purposes.
 - **Memory:** 16 GB per node
 - **CPU:** 4 cores per node

- **Storage:** 200 GB
- **Operating System:** Microsoft Windows Server 2016 (Windows 10) or later, or Red Hat Enterprise Linux 7.5 or later.
- **Clients:**
 - **Memory:** 4 GB
 - **CPU:** 2 cores

It is recommended to use a monitor with full HD (1920x1080) display resolution.

2.2. Software Prerequisites

2.2.1. Server

The Internal Notifications server has the following software requirements:

- RedHat Docker version docker-1.13.1-91 or later
- Kubernetes version 1.13.3-1.15.12
- Helm version 2.16.1 and 3.1.0
- Docker Trusted Registry or a similar supported API/v2 registry (for example, Nexus)

2.2.2. Deployment Server

The Internal Notifications deployment server has the following software requirements:

- Docker CE version 18.06.1 or later
- Docker Trusted Registry or a similar supported API/v2 registry (for example, Nexus)

2.2.3. Clients

Internal Notifications clients have the following software requirements:

- Google Chrome with an enabled JavaScript engine

2.3. Software Delivery Media

Internal Notifications components are delivered with the Interfaces kit, solution artifacts, and configuration files:

- **ADMS_interfaces.msi:** Includes the Data Correction Interface and ADMS interface servers, including the Core Notification Server.

- **msgFormat.json:** Optionally configures Internal Notifications messages.
- **internal-notifications-values.yaml:** Defines the deployment settings for the Internal Notifications. Configures the Internal Notifications application and database settings.
- **foundation-base.yaml:** Defines the deployment settings for the Foundation Base. Configures the security features and other functionality provided by Foundation.
- **internal-notification-cache-<TAG>.tar.gz:** Cache container with all artifacts needed for Internal Notifications deployment in environments without Internet connection or without access to GE's DTR (Docker Trusted Registry). This container includes Internal Notification services, PostgreSQL database, and the Core Notification Consumer application.
- **internal-notifications-<>.tgz:** Helm chart package of internal notification.

Note

GE has reviewed the critical Common Vulnerabilities and Exposures (CVEs) found in the Internal Notifications cache container and our applications are not impacted by these CVEs.

- **foundation-base-cache-<TAG>.tar.gz:** Cache container with all artifacts needed for Foundation Base deployment in environments without Internet connection or without access to GE's DTR (Docker Trusted Registry).
- **nexus-bundle.tgz:** Installation package for installing Nexus.
- **CoreNotificationConsumerConfig.xml:** Configuration file for the Core Notification Consumer application, which includes connection, certificate, and logging settings.
- **privateSettings.json:** Database, UAA, SMTP, and LDAP connection details for the Internal Notification services.

3. Setting Up a Deployment Server (Air Gapped)

! Important

Skip this section if you have access to the GE's Docker Registry, DTR.

If the target environment is air gapped (does not have an Internet connection or access to GE's Docker Registry, DTR), a cache container is transferred to the target environment first, and then run to populate the Nexus with the cached artifacts for that version of the solution. The cache container can then be used to deploy or upgrade the air-gapped environment, which instructs the deployment targets to pull all component artifacts directly from the Nexus (rather than attempting to connect to the DTR).

The Cache Container (`internal-notification-cache-<TAG>.tar`) and Helm chart (`internal-notifications-<>.tgz`) contains all artifacts needed for Internal Notifications deployment.

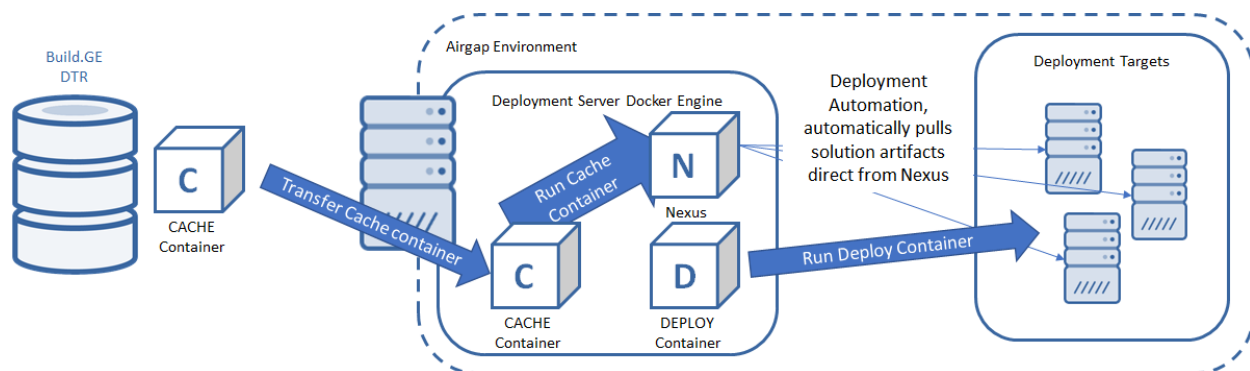


Figure 5. Deployment in an Air Gapped Environment

3.1. Installing the Nexus Repository

i Note

Skip this section if you already have Nexus.

To deploy the Nexus repository manually:

1. Install docker-ce. The version must be above 17.12.0.
2. Install docker-compose:

```
sudo yum install wget -y
sudo wget https://github.com/docker/compose/releases/download/1.21.0/docker-compose-$(uname -s)
-$(uname -m) -O /usr/local/bin/docker-compose
```

3. Copy the `nexus-bundle.tgz` file to a local folder.

4. Untar the bundle:

```
tar -xzf nexus-bundle.tgz
cd bundle
```

5. Run the `start.sh` script, passing the IP address of the Nexus:

```
sudo ./start.sh -i <nexus_ip> -p /tmp/certs_tmp/ -n cert.crt -t airgap
```

For example:

```
sudo ./start.sh -i 192.168.2.11 -p /tmp/certs_tmp -n cert.crt -t airgap
```

Note

If you somehow run `start.sh` with no options or with a bad overall configuration, replace 'start.sh' with 'reset.sh'. This will help reset the files to a clean state.

```
sudo ./reset.sh -i <nexus_ip> -p /tmp/certs_tmp/ -n cert.crt -t <prime|airgap>
```

6. Check your Nexus at `https://<server_ip>`

The Nexus installation creates certificates at `/tmp/certs_tmp/`. It also creates two repositories. You will need them for deployment.

7. Verify the repository configuration:

```
Name: docker-group
Format: docker
Type: hosted
Port: 8082 (https)
---
Name: bsf-generic-virtual
Format: raw
Type: hosted
```

8. If the Nexus repositories do not exist, create them manually.

9. Verify you can log in to Docker:

```
docker login <Nexus_IP:<port>
```

3.2. Configuring Nexus for Kubernetes Nodes

To configure the HTTPS Docker registry:

1. Copy the Nexus pem certificate from Nexus to each Kubernetes node.

```
scp /opt/registry/keys/<Nexus_IP_Address>.crt \ root@<Kubernetes_Node_IP_Address>:/etc/pki/ca-
```

```
trust/source/anchors/
```

i Note

Execute this command for each Kubernetes node.

2. Add the self-signed certificate to the trust store.

```
update-ca-trust
```

i Note

Execute this command for each Kubernetes node.

3. Add a trusted Airgap registry to Docker Daemon.

```
mkdir -p /etc/docker/certs.d/<Nexus_IP_Address>\:<Nexus_Port>  
cp -v /etc/pki/ca-trust/source/anchors/<Nexus_IP_Address>.cert  
/etc/docker/certs.d/<Nexus_IP_Address>\:<Nexus_Port>/ca.crt
```

i Note

Execute this command for each Kubernetes node.

4. Run docker login to add sign-in information to daemon.json.

```
docker login <Nexus_IP_Address>:<Nexus_Port>
```

i Note

Execute this command for each Kubernetes node.

5. Enable kubelet to authenticate with Nexus.

```
scp /root/.docker/config.json root@< Kubernetes_Node_IP_Address >:/var/lib/kubelet/
```

i Note

Execute this command for each Kubernetes node.

6. Restart Docker.

```
systemctl daemon-reload  
systemctl stop kubelet  
systemctl stop docker  
systemctl start docker  
systemctl start kubelet
```

If you cannot set up an HTTPS connection between the Nexus and Kubernetes nodes, configure the Nexus Docker Registry as insecure in the `daemon.json` file on each Kubernetes node. (This option is not recommended.)

```
# Add to file:
vi /etc/docker/daemon.json
{
  "insecure-registries": [<Nexus_IP>:8082"]
}
```

3.3. Uploading Artifacts to Nexus

To upload artifacts to Nexus:

1. Load the tarball into your local Docker Registry:

```
docker load -i internal-notification-cache-<TAG>.tar
docker load -i foundation-base-cache-<TAG>.tar
docker load -i .tar pgpool-<TAG>.tar
docker load -i .tar kube-webhook-certgen-<TAG>.tar
docker load -i .tar ingress-nginx-controller-<TAG>.tar
```

2. Tag an image:

```
docker tag <image ID> <Nexus IP>:<Port>/internal-notification-cache:<Tag>
docker tag <image ID> <Nexus IP>:<Port>/foundation-base-cache:<Tag>
docker tag <image ID> <Nexus IP>:<Port>/pgpool:<Tag>
docker tag <image ID> <Nexus IP>:<Port>/kube-webhook-certgen:<Tag>
docker tag <image ID> <Nexus IP>:<Port>/ingress-nginx-controller:<Tag>
```

For example:

```
docker tag registry.gear.ge.com/de-adms/internal-notification-cache:latest 192.168.2.11:8082/de-
adms/internal-notification-cache:latest
```

3. Upload the Foundation Base artifacts to Nexus:

```
docker run --env DOCKER_HOST=<IP of Exposed Docker Daemon> <Nexus IP>:<Port>/foundation-base-
cache:<Tag> python<Version> post_artifacts.py -u <Nexus_Username> -p <Nexus_Password> -n
<Nexus_IP> -d <Nexus_Docker_Repo_Port> --raw-repo-id <Raw_Repo_Id>
```

Manually push additional images to Nexus server:

```
docker push <Nexus IP>:<Port>/ingress-nginx-controller:<Tag>
docker push <Nexus IP>:<Port>/kube-webhook-certgen:<Tag>
docker push <Nexus IP>:<Port>/pgpool:<Tag>
```

For example:

```
docker run --env DOCKER_HOST=tcp://192.168.2.2:2375 192.168.2.11:8082/de-adms/foundation-base-cache:latest python3.7 post_artifacts.py -u admin -p admin123 -n 192.168.2.11 -d 8082 --raw-repo -id bsf-generic-virtual
```

4. For Internal Notifications specifically, create a `config.json` file with the password of the Nexus server.
5. Upload Internal Notifications artifacts to Nexus:

```
docker run --env DOCKER_HOST=<IP of Exposed Docker Daemon -v /root/.docker/config.json:/root/.docker/config.json <Nexus IP>:<Port>/internal-notification-cache:<Tag> python3 post_artifacts.py -r <Nexus_Docker_Repo_Port> -v 5
```

6. Verify that Docker images have been loaded:

```
docker images
```

4. Setting Up Internal Notifications

The Internal Notifications setup process includes the following steps:

- Configuring the Foundation Base components
- Configuring the Internal Notifications components
- Deploying the Foundation Base and Internal Notifications components
- Configuring the Core Notification Server
- Configuring the Core Notification Consumer
- Setting up the database

4.1. Configuring Foundation Base Components

The Internal Notifications solution uses Foundation Base, which includes security components of Foundation. For more information about Foundation, consult your GE project team.

The Foundation Base services use cryptographic keys and certificates in these areas:

- The Ingress Controller to perform TLS termination.
- Public/Private key pair for UAA to sign access tokens.
- Trusted Certificates for LDAPS clients to validate the authenticity of the LDAPS server. (You can skip this step if your LDAPS server uses a certificate signed by a trusted certificate authority.)

These keys and certificates must be created *prior to* deploying Foundation Base.

4.2. Configuring Internal Notifications Components

4.2.1. Preparing Local Setup Folders

To prepare the setup folders:

1. Create a local folder for deployment (hereinafter, <App_Folder>). You will deploy the Internal Notifications application from this folder.
2. Copy the `internal-notifications.yaml` and `foundation-base.yaml` files to <App_Folder>.
3. Optionally, copy the `msgFormat.json` file to <Notifications_Configuration_Folder>.
4. Copy the `CoreNotificationConsumerConfig.xml` file to <Notifications_Configuration_Folder>.
5. Copy the `admin.conf` file that your Kubernetes instance uses from `/etc/kubernetes` to <Notifications_Configuration_Folder>.
6. Create the `cert` folder in the <Notifications_Configuration_Folder>.

7. Copy the `internal-notifications-uaa-client.yaml` to the directory

A sample folder structure that is used for deployment is shown below. This structure can be used as a reference to create the folder structure.

```
├── k8s-base-manifest.yaml
├── Foundation helm chart
├── Internal Notifications helm chart
├── apps
│   ├── Custom-namespace.yaml
│   ├── foundation-base.yaml
│   ├── foundation-data.yaml
│   ├── foundation-osb.yaml
│   ├── foundation-ui-server.yaml
│   └── internal-notifications.yaml
├── apps-config
└── internal-notifications-values.yaml
```

In the example above, the Helm charts folder contains all the required files for deploying the Foundation Base and Internal Notifications Helm chart.

The following table describes files specific to the Internal Notifications functionality.

Table 1. Configuration Folder Contents for Internal Notifications

File	Folder	Description	Required by YAML files
<code>foundation-base.yaml</code>	<code>./</code>	The Helm chart yaml file for deploying Foundation Base components.	Foundation Base
<code>internal-notifications.yaml</code>	<code>./</code>	The Helm Chart yaml file for deploying Internal Notifications components.	Internal Notifications
<code>admin.conf</code>	<code>./local</code>	Provides access to the Kubernetes cluster.	All
<code>msgFormat.json</code>	<code>./local</code>	Optional file to adjust the default Internal Notification message formats.	Internal Notifications
<code>privateSettings.json</code>	<code>./local</code>	Optional file to provide the connection details of the database, UAA, SMTP and JWT servers.	Internal Notifications
<code>CoreNotificationConsumerConfig.xml</code>	<code>./local</code>	Optional file to define the connection, certificate, and logging settings.	Internal Notifications

4.2.2. Configuring Message Formats

Use the `msgFormat.json` file to customize Internal Notifications message formats.

The default `msgFormat.json` file is shown below:

```
{
  "outputConfig": {
    "DateFormat": "yyyy-MM-dd HH:mm:ss zzz",
```



```

"TextMsgFormat": [
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Subscription: ",
    "VariableName": "subscriptionName",
    "Separator": ";"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Incident: ",
    "VariableName": "incidentName",
    "Separator": ";"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Status: ",
    "VariableName": "incidentStatus",
    "Separator": ";"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "ETR: ",
    "VariableName": "incidentETR"
  }
],
"EmailSubjectFormat": [
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "",
    "TextData": "Notification for Subscription ",
    "VariableName": "subscriptionName"
  }
],
"emailBodyFormat": [
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "VariableName": "notificationSummary",
    "Separator": "<br /><br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Incident Name: ",
    "VariableName": "incidentName",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",

```

```

    "VariableEmptyText": "N/A",
    "TextData": "Incident Start Time: ",
    "VariableName": "eventStartTime",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Incident Status: ",
    "VariableName": "incidentStatus",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "ETR: ",
    "VariableName": "incidentETR",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Number of Affected Customers: ",
    "VariableName": "affectedCustomers",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Device Name: ",
    "VariableName": "outageDeviceName",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Device Type: ",
    "VariableName": "outageDeviceType",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Feeder ID: ",
    "VariableName": "outageFeederId",
    "Separator": "<br />"
  },
  {
    "IfVariableEmpty": "print",
    "VariableEmptyText": "N/A",
    "TextData": "Substation ID: ",
    "VariableName": "outageSubstationId",
    "Separator": "<br />"
  }

```

```

},
{
  "IfVariableEmpty": "skip",
  "TextData": "ServiceCenter",
  "VariableName": "outageServiceCenter",
  "Separator": "<br />"
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone1_description",
  "Separator": ": "
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone1_name",
  "Separator": "<br />"
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone2_description",
  "Separator": ": "
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone2_name",
  "Separator": "<br />"
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone3_description",
  "Separator": ": "
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone3_name",
  "Separator": "<br />"
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone4_description",
  "Separator": ": "
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone4_name",
  "Separator": "<br />"
},
{
  "IfVariableEmpty": "skip",
  "VariableName": "zone5_description",
  "Separator": ": "
},
{

```

```

        "IfVariableEmpty": "skip",
        "VariableName": "zone5_name",
        "Separator": "<br />"
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone6_description",
        "Separator": ": "
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone6_name",
        "Separator": "<br />"
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone7_description",
        "Separator": ": "
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone7_name",
        "Separator": "<br />"
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone8_description",
        "Separator": ": "
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone8_name",
        "Separator": "<br />"
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone9_description",
        "Separator": ": "
    },
    {
        "IfVariableEmpty": "skip",
        "VariableName": "zone9_name",
        "Separator": "<br />"
    }
    ]
}
}

```

4.2.3. Configuring the Core Notification Consumer

Use the `CoreNotificationConsumerConfig.xml` file to configure the Core Notification Consumer

connection, certificate, and logging settings. For example:

```
<?xml version="1.0" encoding="utf-8"?>
<NotificationConsumerConfig>
  <WebServiceBaseUrl><Kubernetes_Cluster_URL></WebServiceBaseUrl>
  <CertificateSubject><Certificate_Subject_Name></CertificateSubject>
  <LogFileName>\\log\\CoreNotificationConsumer.txt</LogFileName>
  <MaxMessagesInLogFile>10000</MaxMessagesInLogFile>
  <LogLevel>5</LogLevel>
</NotificationConsumerConfig>
```

4.2.4. Generating Certificates

Secure connections among the Internal Notifications and ADMS systems is established using an HTTPS connection with TLS termination.

For HTTPS with TLS, you need to obtain SSL certificates from your certificate authority. The certification authority is based on Active Directory or other services. Therefore, you need to have two certificates: one for Kubernetes Ingress Controller and another for the ADMS interface server.

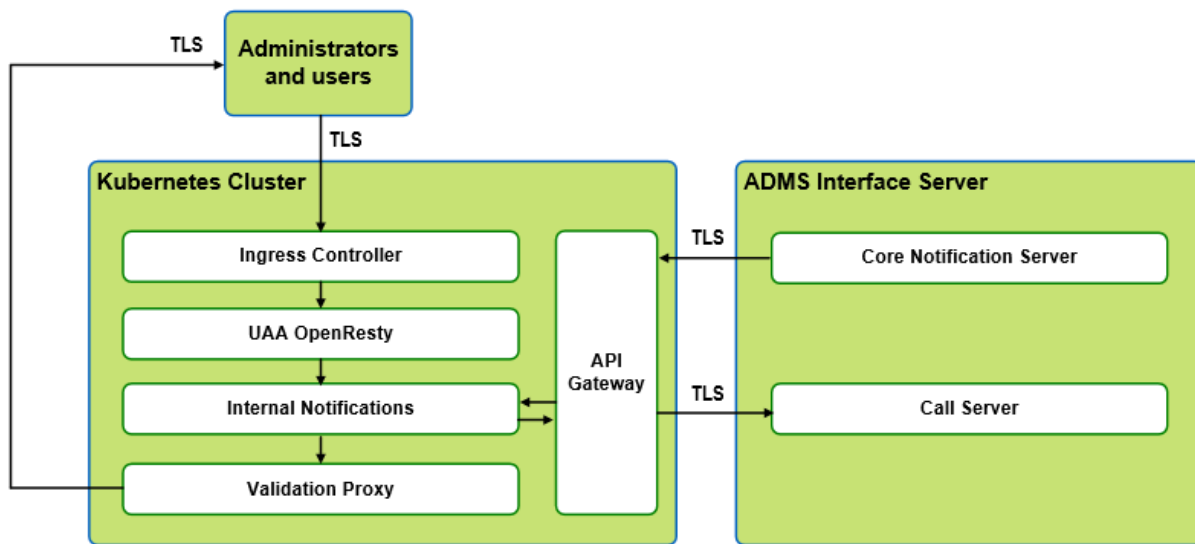


Figure 6. HTTPS connection with TLS Termination between Internal Notifications and ADMS

For Internal Notifications, you need the following certification files:

- Root certificate authority certificate. This certificate is used to issue and sign certificates. (For example, file name = `rootCA.crt`; CN = ADMS Root CA)
- Ingress controller certificate and key. These files are used for establishing connections with Active Directory. (For example, file name = `ingress-local-tls.crt` and `ingress-local-tls.key`; `ingress.local`)
- ADMS server PFX file with a certificate and the associated private key. This file is used for secure communication between the Kubernetes cluster and ADMS interface servers. (For example, file

```
name = adms.pfx; CN = ADMS)
```

For production, it is recommended to use certificates issued by a certificate authority. For testing and demo purposes, you can use self-signed certificates.

The example below shows how to generate OpenSSL certificates using the `genrsa` tool. This is a standard tool for most Linux distributions. For more information, see <https://www.openssl.org/source/>.

To generate self-signed certificates:

1. Generate a root certificate authority certificate and key:

```
openssl genrsa -out rootCA.key 2048
openssl req -x509 -new -key rootCA.key -days 10000 -out rootCA.crt
```

2. Generate a certificate and key for Ingress Controller:

Note

It is recommended to set CN to the actual ingress controller DNS host name (web address you access when connecting to a secure site).

```
openssl genrsa -out ingress-local-tls.key 2048
openssl req -new -key ingress-local-tls.key -out ingress-local-tls.csr
openssl x509 -req -in ingress-local-tls.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out
ingress-local-tls.crt -days 5000
```

3. Generate a certificate and key for ADMS servers:

Note

It is recommended to set CN to the actual ADMS DNS host name (web address you access when connecting to a secure site).

```
openssl genrsa -out adms.key 2048
openssl req -new -key adms.key -out adms.csr
openssl x509 -req -in adms.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out adms.crt
-days 5000
```

4. Convert the ADMS server certificate and key to the pfx format:

```
openssl pkcs12 -export -out cert.pfx -inkey adms.key -in adms.crt -certfile adms.crt
```

5. Copy the content of `adms.crt` certificate in yaml file of internal notification yaml file.

4.2.5. Internal Notifications: Required Permissions and Roles

To use the **Internal Notifications** application, specific permissions and roles must be configured in the **Active Directory**.

4.2.5.1. Roles

The roles below define the aggregation of permissions for different user groups:

- roleName: Internal Notifications Viewer
description: Can access internal notifications UI
- roleName: Internal Notifications User
description: Can access internal notifications UI and create subscriptions
- roleName: Internal Notifications Administrator
description: Can access internal notifications UI and manage subscriptions

4.2.5.2. Permissions

The following permissions control access to Internal Notifications functionalities:

- permissionName: internalNotifications.subscriptions.write.writeAll
description: Can view and delete subscriptions for all users, and enable or disable notifications.
- permissionName: internalNotifications.subscriptions.read.readAll
description: Can view subscriptions for all users.
- permissionName: internalNotifications.mySubscriptions.write.writeAll
description: Can view, modify, and delete subscriptions for the current user.
- permissionName: internalNotifications.mySubscriptions.read.read
description: Can view subscriptions for the current user.

4.2.5.3. Roles Mappings

- name: Internal Notifications Viewer
permissions: [
 'internalNotifications.subscriptions.read.readAll',
 'internalNotifications.mySubscriptions.read.read'
]
- name: Internal Notifications User
permissions: [
 'internalNotifications.subscriptions.read.readAll',
 'internalNotifications.mySubscriptions.write.writeAll',
 'internalNotifications.mySubscriptions.read.read'
]
- name: Internal Notifications Administrator

```
permissions: [
  'internalNotifications.subscriptions.write.writeAll',
  'internalNotifications.subscriptions.read.readAll',
  'internalNotifications.mySubscriptions.write.writeAll',
  'internalNotifications.mySubscriptions.read.read'
]
```

4.2.5.4. Usage Notes

Permissions ensure fine-grained control over the Internal Notifications application. The application currently supports the following permissions:

- Users with the `internalNotifications.subscriptions.write.writeAll` permission are considered **Administrators**.
- Users with the `internalNotifications.mySubscriptions.write.writeAll` permission can **subscribe to notifications**.

4.2.5.5. Default Role Assignments in Foundation Deployment

In the default **foundation deployment**, the following user groups are assigned Internal Notifications roles:

1. Supervisors

- Assigned Role: `Internal Notifications Administrator`
- Privileges: Users in this group can act as Administrators and subscribe to notifications.

2. Operators

- Assigned Role: `Internal Notifications User`
- Privileges: Users in this group can subscribe to notifications.

3. Service Account Group 5

- Assigned Role: `Internal Notifications Administrator`
- Description: This group represents client authority defined in the UAA client. It enables the `CoreNotificationServer` to connect to the Kubernetes cluster and access Internal Notifications Services.

```
usergroups:
- name: Operators
  roles: [
    'Operator',
    'Session Viewer',
    'Dashboard Viewer',
    'Internal Notifications User'
  ]

- name: Supervisors
```



```

roles: [
  'Supervisor',
  'Kibana Viewer',
  'Grafana Viewer',
  'Prometheus Viewer',
  'Alertmanager Viewer',
  'Kiali Viewer',
  'Jaeger Viewer',
  'Policy Report Viewer',
  'Dashboard Administrator',
  'User Details Viewer',
  'Internal Notifications Administrator'
]

- name: Service Account Group 5
  roles: [
    'Tile Service Consumer',
    'AORs Manager Sessions Consumer',
    'ES Adapter Service Consumer',
    'Preferences Service Consumer',
    'Internal Notifications Administrator'
  ]

```

This configuration ensures that users and service accounts have the appropriate access levels to manage and utilize Internal Notifications effectively.

4.2.6. Configuring the YAML Files

The YAML file defines application settings and deployment for both the Internal Notifications and Foundation Base components.

Configure the Custom-namespace.yaml, foundation-base.yaml and internal-notifications.yaml files in <App_Folder> as follows:

1. Update the custom namespace yaml file by adding internal notification namespace as **etd** and enable the istio_injected flag
2. Configure IPs according to your current configuration.
3. Specify the address of the Docker Registry that contains YAML file (GE's DTR or a Nexus registry on a deployment server).
4. When deploying from a Nexus Docker Registry on a deployment server, update the docker "image" paths to use the Nexus host and port.
5. Configure the deployment settings, such as paths to the Docker registry, container names, and components to be deployed.
6. Set the actual Local Persistent Volume storage names in the storageClass properties.
7. Configure the Foundation Base settings, including Infispan, Patroni, and LDAP.

4.2.6.1. Configuring the Internal Notifications YAML File

Below table's describes the configuration settings for the Internal Notifications YAML file.

4.2.6.2. Internal Notifications Consumer

Field	Default Value	Description
<code>internal-notifications-consumer.image.repository</code>	<code>internal_notifications/internal-notifications-consumer</code>	Repository for the Internal Notifications consumer image.
<code>internal-notifications-consumer.image.tag</code>	<code>16.18.0-rc.4</code>	Tag for the internal notifications consumer image. This parameter is for reference only and should not be changed.

4.2.6.3. Internal Notifications Service

Field	Default Value	Description
<code>internal-notifications-service.replicaCount</code>	<code>1</code>	Number of replicas for the service.
<code>internal-notifications-service.image.repository</code>	<code>internal_notifications/internal-notifications-service</code>	Repository for the service image.
<code>internal-notifications-service.image.tag</code>	<code>16.18.0-rc.8</code>	Tag for the internal notifications service image. This parameter is for reference only and should not be changed
<code>internal-notifications-service.privateSettings.databaseConfig.notificationPollIntervalSeconds</code>	<code>60</code>	Polling interval for notifications in seconds.
<code>internal-notifications-service.privateSettings.databaseConfig.dbKeyStorageLocation</code>	<code>/home/notify/.notify/data</code>	Path to store database keys. This parameter is for reference only and should not be changed
<code>internal-notifications-service.privateSettings.jwtConfig.authority</code>	<code>http://sws-uaa.foundation-cluster-zerotrust.svc.cluster.local:8080/uaa/</code>	Authority URL for JWT configuration. This parameter is for reference only and should not be changed.
<code>internal-notifications-service.privateSettings.jwtConfig.audience</code>	<code>apigatewayclientid</code>	Audience for JWT.
<code>internal-notifications-service.privateSettings.jwtConfig.issuer</code>	<code>http://sws-uaa.foundation-cluster-zerotrust.svc.cluster.local:8080/uaa/oauth/token/</code>	JWT token issuer. This parameter is for reference only and should not be changed.
<code>internal-notifications-service.privateSettings.jwtConfig.usernameClaimType</code>	<code>user_name</code>	Username claim type for JWT. This parameter is for reference only and should not be changed.
<code>internal-notifications-service.persistentVolume.size</code>	<code>5G</code>	Persistent volume size.
<code>internal-notifications-service.persistentVolume.storageClass</code>	<code>local-storage</code>	Storage class for persistent volume.

4.2.6.4. Internal Notifications UI

Field	Default Value	Description
internal-notifications-ui.image.repository	internal-notifications-ui	Repository for the UI image.
internal-notifications-ui.image.tag	16.18.0-rc.11	Tag for the UI image. This parameter is for reference only and should not be changed
internal-notifications-ui.callServerExternalGatewayRoute.endpoints	https://{WIN_SERVER}:8443	External gateway route endpoints for the server. Replace {WIN_SERVER} with the Server IP where Call Server is running.
internal-notifications-ui.admsExternalGateway.loggingLevel	ERROR	Logging level for ADMS external gateway.
internal-notifications-ui.admsExternalGateway.replicas	1	Number of replicas for the ADMS external gateway.
internal-notifications-ui.admsCertSecret.cert	<certificate-content>	Certificate for ADMS external gateway.

4.2.6.5. Global Parameters

Field	Default Value	Description
global.image.registry	dig-grid-artifactory.apps.ge.com/virtual-docker	Global image registry.
global.image.pullPolicy	Always	Global image pull policy.
global.TZ	UTC	Timezone for the application.
global.imagePullSecrets	docker-registry	Image pull secret for the global registry.
global.zeroTrustNamespace	foundation-cluster-zerotrust	Namespace for zero trust configuration. This parameter is for reference only and should not be changed.
global.clusterExternalUrl	http://ingress.local	External URL for the cluster. Replace with the kubernetes external url.
global.fqdnSuffix	.svc.cluster.local	Fully qualified domain name suffix for services. This parameter is for reference only and should not be changed.
global.jwksUri	http://sws-uaa.foundation-cluster-zerotrust.svc.cluster.local:8080/uaa/token_keys	JWKS URI for JWT configuration. This parameter is for reference only and should not be changed.
global.zitadel.enabled	false	Enables or disables Zitadel integration. The current release does not include support for Zitadel. This parameter should be set to false.
global.zitadel.organisationName	foundation	Organisation name for Zitadel.

Field	Default Value	Description
<code>global.smtp.createSecret</code>	<code>true</code>	Indicates whether an SMTP secret should be created. If an SMTP secret is being created prior to installing Internal Notifications, set this parameter to <code>false</code> .
<code>global.smtp.secretName</code>	<code>smtp-internal-notifications-secret</code>	Name of the SMTP secret. If you create an SMTP secret before installing Internal Notifications, make sure to use this name for the secret.
<code>global.encryptionKey.createSecret</code>	<code>true</code>	Indicates whether an Encryption key secret should be created. If an Encryption key secret is being created prior to installing Internal Notifications, set this parameter to <code>false</code> .
<code>global.encryptionKey.secretName</code>	<code>internal-notifications-key-secret</code>	Name of the Encryption key secret. If you create an Encryption key secret before installing Internal Notifications, make sure to use this name for the secret.
<code>global.postgres.serviceHost</code>	<code>postgres-db-pg-cluster.foundation-cluster-zerotrust.svc.cluster.local</code>	Host for PostgreSQL service.
<code>global.postgres.servicePort</code>	<code>5432</code>	Port for PostgreSQL service.
<code>global.postgres.showSQL</code>	<code>false</code>	Indicates whether SQL queries are logged.
<code>global.postgres.inDbName</code>	<code>notifya</code>	Database name for notifications. This parameter is for reference only and should not be changed.
<code>global.postgres.inCreateSecret</code>	<code>true</code>	Indicates whether to create a database credentials secret. If the postgres credentials secret is being created prior to installing Internal Notifications, set this parameter to <code>false</code> .
<code>global.postgres.inDbSecret</code>	<code>pg-internal-notifications-secret</code>	Name of the database secret. If you create the database credential secret before installing Internal Notifications, make sure to use this name for the secret.
<code>global.postgres.servicePool</code>	<code>true</code>	Enables service pooling for PostgreSQL.
<code>global.postgres.sslMode</code>	<code>Require</code>	SSL mode for PostgreSQL.
<code>global.postgres.trustServerCertificate</code>	<code>true</code>	Indicates whether to trust the server certificate for SSL. This parameter is for reference only and should not be changed.
<code>global.postgres.sampleDataCreation</code>	<code>false</code>	Indicates whether to create sample data in the database.
<code>global.generatePasswordSecret</code>	<code>true</code>	Indicates whether a UAA client secret should be generated. The UAA client is used by the CoreNotificationServer to connect with the Kubernetes Cluster.
<code>global.internalNotifications.clientId</code>	<code>internal-notification-app-admin</code>	Client ID for internal notifications. This parameter is for reference only and should not be changed.

Field	Default Value	Description
<code>global.internalNotifications.clientSecretName</code>	<code>internal-notification-app-admin</code>	Client secret name for internal notifications. If the UAA client secret is being created before installing Internal Notifications, make sure to use this name for the secret.
<code>global.internalNotifications.client.authorities</code>	<code>service.account.group5</code>	Client authorities for internal notifications. This parameter is for reference only and should not be changed.

4.2.6.6. Creating secrets for Internal Notifications

The following secrets are required for Internal Notifications:

- Database Credentials
- SMTP settings
- UAA Client Secret
- Encryption key Secret

The user can choose to create these secrets manually or let the deployment process create them. Note that all fields in a secret are base64 encoded.

4.3. Database Credentials

The database credentials are stored in the secret `pg-internal-notifications-secret` in the `etd` namespace.

The flag `global.postgres.inCreateSecret` in the `internal-notifications-values.yaml` must be set to `false` to create this secret manually.

Below is an example of the secret where both `username` and `password` are set to the base64-encoded value of the string `apiuser`:

```
apiVersion: v1
kind: Secret
metadata:
  name: pg-internal-notifications-secret
  namespace: etd
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "3"
type: Opaque
data:
  "{{- $password := get (get (lookup "v1" "Secret" .Release.Namespace
    .Values.global.postgres.inDbSecret | default dict) "data" | default dict) "postgresdb-password" |
    default (randAlphaNum 16 | b64enc) }}"
  postgresdb-username: YXBpdXNlcg==
  postgresdb-password: YXBpdXNlcg==
```

4.4. SMTP Settings

The SMTP settings are stored in the secret `pg-internal-notifications-secret` in the `etd` namespace.

The flag `global.smtp.secretName` in the `internal-notifications-values.yaml` must be set to `false` to create this secret manually.

Configure the SMTP server for sending SMS and email notifications.

The `smtp-text-message-sender` value should be the address of a gateway that converts email messages to text messages. This will typically be the address of a third-party SMTP to SMS gateway capable of sending messages to phone numbers on any service provider network. However, if all phone numbers being used are on the same service provider network, it is possible to use the address of the public SMTP gateway for that service provider.

For example, if all subscriber devices are on the AT&T network, `textMessageSender` could be set as: `textMessageSender: "@txt.att.net"`

Below is an example of the secret for SMTP settings:

```
apiVersion: v1
kind: Secret
metadata:
  name: smtp-internal-notifications-secret
  namespace: etd
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "3"
type: Opaque
data:
  smtp-server: c210cC5ncmlkLmRpZ2l0YWwuz2UuY29t # smtp.griddigital.com
  smtp-port: MjU= # 25
  smtp-authentication-required: ZmFsc2U= # false
  smtp-ssl-required: ZmFsc2U= # false
  smtp-username: ""
  smtp-password: ""
  smtp-from-address: aW50ZXJuYWwubm90aWZpY2F0aW9uc0BnZS5jb20= # internal.notifications@ge.com
  smtp-from-name: SW50ZXJuYWwuz2UuY29tF0aW9ucw== # Internal Notifications
  smtp-text-message-sender: QGR1bW15bWVzYWdpbmcuZHVtbXkuY29tLnVh # @dummymessaging.dummy.com.ua
```

4.5. UAA Client Secret

The UAA Client secret is stored in the secret `internal-notification-app-admin` in the `etd` namespace.

The flag `global.generatePasswordSecret` in the `internal-notifications-values.yaml` must be set to `false` to create this secret manually.

Below is an example of the secret for UAA Client with the secret value set to the base64-encoded value of the string `internal-notification-app-admin-secret`:

```

apiVersion: v1
kind: Secret
metadata:
  name: internal-notification-app-admin
  namespace: etd
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "3"
type: Opaque
data:
  secret: aW50ZXJuYWwtbm90aWZpY2F0aW9uLWFWcC1hZG1pb1ZWNyZXQ=

```

4.6. Encryption key Secret

The Encryption key secret is stored in the secret `internal-notifications-key-secret` in the `etd` namespace. The encryption key is used to encrypt and decrypt the sensitive data like email and phone number while storing and retrieving from the database.

The flag `global.encryptionKey.createSecret` in the `internal-notifications-values.yaml` must be set to `false` to create this secret manually.

Note

The encryption key is associated with the location `/home/notify/.notify/data` in the `internal-notification-service` pod.

If upgrading from an existing deployment, ensure that the same key is used when creating the secret to prevent data corruption.

Below is an example of the secret for Encryption key with the secret value set to the base64-encoded value of the below encryption key

```

xml
<?xml version="1.0" encoding="utf-8"?>
<key id="546d69e1-bcdc-435a-ab07-e95e889728df" version="1">
  <creationDate>2025-02-19T12:03:39.8309377Z</creationDate>
  <activationDate>2025-02-19T12:03:39.8256666Z</activationDate>
  <expirationDate>2025-05-20T12:03:39.8256666Z</expirationDate>
  <descriptor
deserializerType="Microsoft.AspNetCore.DataProtection.AuthenticatedEncryption.ConfigurationModel.Authen
ticatedEncryptorDescriptorDeserializer, Microsoft.AspNetCore.DataProtection, Version=8.0.
0.0, Culture=neutral, PublicKeyToken=adb9793829dda60">
    <descriptor>
      <encryption algorithm="AES_256_CBC" />
      <validation algorithm="HMACSHA256" />
      <masterKey p4:requiresEncryption="true"
xmlns:p4="http://schemas.asp.net/2015/03/dataProtection">
        <!-- Warning: the key below is in an unencrypted form. -->

<value>EeGi/Z9N60HwC7gXXkdfb163Z33VuSN0V01t0rJpRwADbCVkU2sgVwWGVamJr1uLiYa179nKYP8uqK9i82oJQQ==</value>

```

```
>
    </masterKey>
  </descriptor>
</descriptor>
</key>
```

The base64-encoded value of the above encryption key is set to `in-encryption-key.xml`.

```
apiVersion: v1
kind: Secret
metadata:
  name: {{ .Values.global.encryptionKey.secretName }}
  namespace: {{ .Release.Namespace }}
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "3"
type: Opaque
data:
  in-encryption-key.xml: |-
```

[illegible]

Example of an YAML File for Custom Namespace

```
namespaces:
- name: foundation-cluster-monitoring
  istio_injected: true
- name: foundation-cluster-operators
  istio_injected: true
- name: foundation-cluster-zerotrust
  istio_injected: true
- name: kyverno
  istio_injected: false
- name: foundation-cluster-tools
  istio_injected: true
- name: foundation-env-default
  istio_injected: true
## add additional namespaces below - the above namespaces are included in the default values file
```



```
and must be present - example below to create conjur namespace with sidecar injection enabled
- name: etd
  istio_injected: true
```

Example of an YAML File for Deploying Foundation Base

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: foundation-base
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  destination:
    namespace: "argocd"
    server: https://kubernetes.default.svc
  project: default
  sources:
    - chart: foundation-base-argocd-wrapper
      repoURL: https://dig-grid-artifactory.apps.ge.com/artifactory/api/helm/foundation-helm
      targetRevision: BASE-VERSION
      helm:
        valueFiles:
          - $valuesRepo/2024-03/foundation-base-argocd/foundation-base-app-values.yaml
    - repoURL: <Git Repo-url>
      targetRevision: FRA-VERSION
      ref: valuesRepo
  syncPolicy:
    automated:
      prune: false
      selfHeal: true
      allowEmpty: true
    syncOptions:
      - CreateNamespace=false
      - RespectIgnoreDifferences=true
```

Foundation Base App yaml file

```
spec:
  destination:
    server: "https://kubernetes.default.svc"
  sources:
    helmRepo: "https://dig-grid-artifactory.apps.ge.com/artifactory/api/helm/foundation-helm"
    values:
      repo: "https://github.build.ge.com/grid-foundation/foundation-reference-app"
      targetRevision: "2024-03"
      foundationProfile:
        prefixPath: "release/foundation-helm-profiles"
        name: "default"

#### sync-wave: 10
```

```

kyvernoRancherCrds:
  chartName: "kyverno-rancher-crds"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 20
foundationPoliciesCerts:
  chartName: "foundation-policies-certs"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 30
zeroTrustOperator:
  chartName: "zero-trust-operator"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 40
istioPrep:
  chartName: "istio-prep"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 40
istiod:
  chartName: "istiod"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 50
foundationOperators:
  chartName: "foundation-operators"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 60
monitoringApps:
  chartName: "monitoring-apps"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 70
k8sIngressNginx:
  chartName: "k8s-ingress-nginx"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 80
zeroTrustApps:
  chartName: "zerotrust-apps"
  #overrideTargetRevision: 1.3.0

#### sync-wave: 81
foundationAuthWS:
  chartName: "foundation-auth-ws"
  #overrideTargetRevision: 1.0.0

#### sync-wave: 90
velero:
  chartName: "velero"
  #overrideTargetRevision: 1.3.0

```

```
#### sync-wave: 100
foundationClusterTools:
  chartName: "foundation-cluster-tools"
  #overrideTargetRevision: 1.3.0
```

Example of an YAML File for Deploying Internal Notifications

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: internal-notifications
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "50"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  destination:
    namespace: etd
    server: https://kubernetes.default.svc
  project: default
  source:
    helm:
      # parameters:
      # - name: global.registry
      #   value: dig-grid-artifactory.apps.ge.com
      valueFiles:
        - ../../apps-config/internal-notifications-values.yaml
    path: charts/internal-notifications
    repoURL: https://github.software.governova.com/grid-operate/internal-notifications-charts.git
    targetRevision: HEAD
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - RespectIgnoreDifferences=true
```

internal-notification-values Yaml file

```
# This is a YAML-formatted file.
# Declare variables to be passed into your templates.
internal-notifications-consumer:
  image:
    repository: internal_notifications/internal-notifications-consumer
    tag: "16.18.0-rc.4"

internal-notifications-service:
  replicaCount: 1
  image:
```

```

    repository: internal_notifications/internal-notifications-service
    tag: "16.18.0-rc.8"
privateSettings:
  databaseConfig:
    notificationPollIntervalSeconds: 60
    dbKeyStorageLocation: "/home/notify/.notify/data"
  jwtConfig:
    authority: "http://sws-uaa.foundation-cluster-zerotrust.svc.cluster.local:8080/uaa/"
    audience: "apigatewayclientid"
    issuer: "http://sws-uaa.foundation-cluster-zerotrust.svc.cluster.local:8080/uaa/oauth/token/"
    usernameClaimType: "user_name"
  persistentVolume:
    size: 5G
    storageClass: local-storage

internal-notifications-ui:
  image:
    repository: internal_notifications/internal-notifications-ui
    tag: "16.18.0-rc.11"
  # Call server api endpoint
  callServerExternalGatewayRoute:
    endpoints:
      - https://10.227.53.16:8443
      # - https://172.227.51.82:8443
      # - https://172.227.51.83:8443
      # - https://172.227.51.84:8443
  admsExternalGateway:
    loggingLevel: ERROR
    replicas: 1
    maxHttpRequestHeaderSize: "1MB"
  admsCertSecret:
    cert: |
      -----BEGIN CERTIFICATE-----
      MIIDozCCAougAwIBAgIUU3KvyEumb+TRqvDf5D9kxCipiKwwDQYJKoZIhvcNAQEL
      BQAwYTELMAGAA1UEBhMCVUxEzARBGNVBAgMC1NvbWU3RhdGUxEzARBGNVBAoM
      CkdFIERpZ210YWwKDAkAMBgNVBAMMH0RFRmI4wIEZvdW5kYXRpb24gQmFzZSA0IFJv
      b3QgQ0EwHhcNMjEwMDY0OTQ0WhcNMzEwMDY0OTQ0WjBhMQswCQYDVQQG
      EwJBVTETMBEGA1UECAwKU29tZS1TdGF0ZTETMBEGA1UECgwKR0UgRGlNaXRhbDEo
      MCYGA1UEAwwfREUyLjAgRm91bmRhdG1vbiBCYXNlIC0gUm9vdCBBDQTCASAwDQYJ
      KoZIhvcNAQEBBQADggEPADCCAQoCggEBA0rcZK4NB0QuunFJ8Di5bhl+PDwP6fDd
      Zdx06w0t0vguq2KagDFJjHa0NyChLHLJQdKHQ+ga2kY4Wqxt1w0LhCvC0gxVYQwV
      IHVinzFD1/KxUGy6n2AIxhcGTLl+pbYL6e54Lsl+T+orN76YYCt72o6cZlpwLh
      ODfTfLX4sP/ljLYJmIw/X0dR85NoDiAVtm2S9pcdl0/ivorJnlq39LJ+LgJgIkG/
      FRJpMil/yqOTwnizq/M9kaM8HCLNuyRzM+m8eIIXCM7hJrp060GWl3ken6BW0QXf
      NPPw9CWpRxTHAZcWsInvoCY+MwD3g9mQ6xHvzbXjgYIoVNMwEqHydkCAwEAAN
      MFewHQYDVR0BBYEFaANoDyDSLFFbNxaBJ7PDKFW0rQ0MB8GA1UdIwQYMBaAFAAN
      oDyDSLFFbNxaBJ7PDKFW0rQ0MA8GA1UdEwEB/wQFMAMBAf8wDQYJKoZIhvcNAQEL
      BQADggEBAEMA0F5u3aiHyfNDJDV3L2me2e1eKTqojr7oMHDNGj0JNqtJ7A1PMwor
      RutqYY6Jwe3j08fuYf9EezS0305LgLS683jkN3KIdwSXlNpDE4aMo1XlTxA+12
      LwbXXLTORMOKP1FQH1kRgaTKKnqCbW1Jet/AyR9DIiPjSjPwVJJm3Pm1ztg4w0e
      jfkDhMu1QiqSQ7aviaC0Kiy8VYiDQLo0GitrFo+C+uaQKXwCAhRIRBZbLnBgFP6Q
      SWtS9o4M0f1tiqCSUEhT8wTYf2pEqPDMtEx5res8sdg+P45cIphFpB7pGojIDzKs
      uVZAiK58024KW49o74I+QrMbWqeH0mc=

```

```

-----END CERTIFICATE-----

internal-notifications-schema-job:
  image:
    pullPolicy: IfNotPresent
    repository: spilo-cdp
    tag: 14-2.1-p6-build.7

global:
  image:
    registry: dig-grid-artifactory.apps.ge.com/virtual-docker
    pullPolicy: Always
  TZ: "UTC"
  imagePullSecrets:
    - name: docker-registry
  zeroTrustNamespace: foundation-cluster-zerotrust
  clusterExternalUrl: http://ingress.local
  fqdnSuffix: .svc.cluster.local
  jwksUri: http://sws-uaa.foundation-cluster-zerotrust.svc.cluster.local:8080/uaa/token_keys
  serviceProxyDns: ""
  zitadel:
    enabled: true
    organisationName: "foundation"
  smtp:
    createSecret: true
    secretName: smtp-internal-notifications-secret
  encryptionKey:
    createSecret: true
    secretName: internal-notifications-key-secret
  postgres:
    serviceHost: postgres-db-pg-cluster.foundation-cluster-zerotrust.svc.cluster.local
    servicePort: 5432
    showSQL: false
    dbName: "notifya"
    inCreateSecret: true
    inDbSecret: pg-internal-notifications-secret
    servicePool: true
    sslMode: "Require"
    trustServerCertificate: true
    sampleDataCreation: 'false'
  generatePasswordSecret: true
  internalNotifications:
    clientId: internal-notification-app-admin
    clientSecretName: internal-notification-app-admin
    client:
      authorities: # Needs to match etd group defined in roles-config YAML
        - service.account.group5

```

4.6.1. Configuring the hosts.etc File

Important

Perform the following operation for each machine (host) where ADMS interface servers (OMS Interface Server and Core Notification Server) are running. Also perform the following operation for Internal Notification client machines.

In the `hosts.etc` file, add the following entry:

```
<Kubernetes_Cluster_IP> service.ingress.local zitadel.ingress.local admin.ingress.local  
a6dashboard.ingress.local
```

After this configuration, you can use `ingress.local` instead of your Kubernetes host name in all configurations. After migrating to Zitadel UAA, the URL changes to <https://zitadel.ingress.local/oauth/v2/token> and the service is accessed via <https://service.ingress.local>.

4.6.2. Setting Up Firewall Rules

Open the firewall ports listed in the following table to allow communication among the Kubernetes cluster, ADMS servers, and LDAP/AD server.

Table 2. Firewall Ports to Open

Source	Destination	Port	Protocol
Core Notification Consumer (Kubernetes Cluster)	Core Notification Server (ADMS Interface Server)	18802	HTTPS
Kubernetes Cluster	LDAP/AD Server	Project-specific (by default, 636)	LDAPS
Core Notification Server (ADMS Interface Server)	Core Notification Consumer (Kubernetes Cluster)	30001	HTTPS/WSS
Internal Notification Service (Kubernetes Cluster)	Call Server (ADMS Interface Server)	8443	HTTPS
ADMS Interface Server	Kubernetes Cluster	443	HTTPS/WSS

Note

Port numbers can be configured in configuration files for the associated applications.

4.6.3. Importing Certificates to the ADMS Hosts

Important

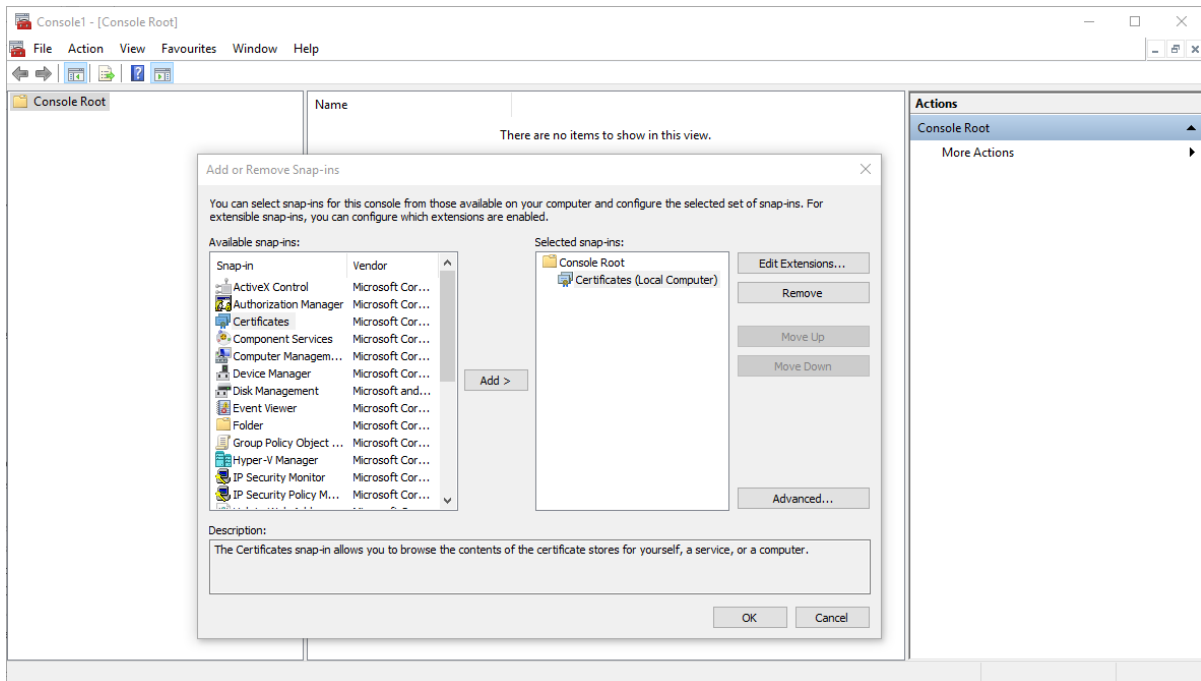
Perform the following steps for each machine (host) where ADMS interface servers (OMS Interface Server and Core Notification Server) are running.

To import certificates to an ADMS interface server host:

1. Copy the Root certificate authority certificate (`rootCA.crt`) and the ADMS server PFX file with a

certificate and the associated private key (adms.pfx) to the ADMS machine.

2. Open the Microsoft Management console.
3. From the File menu, select Add/Remove Snap-in.
4. From the available snap-ins, select Certificates and click Add.
5. Select the Computer Account option and click Next.
6. Select the Local Computer option and click Finish.

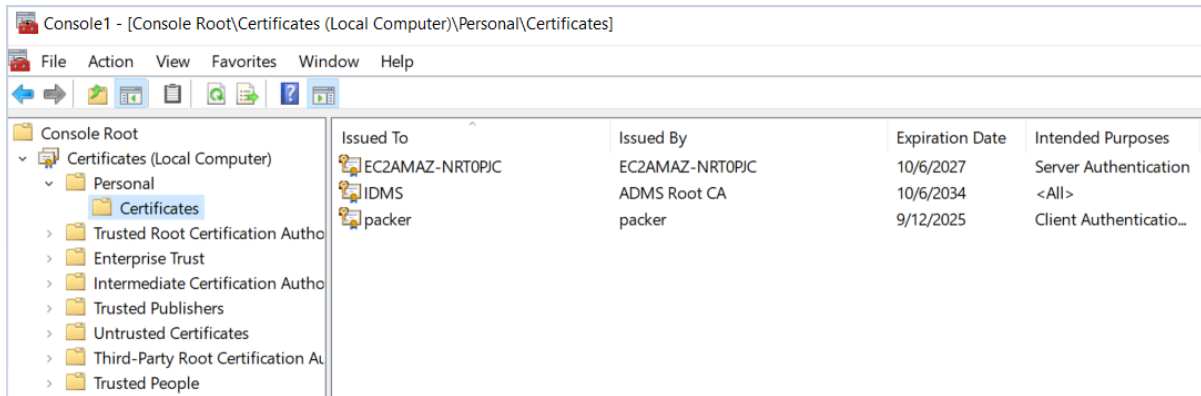


7. Click OK.
8. In Console Root, expand the Certificates (Local Computer) list.
9. Right-click Personal, select All Tasks, and click Import.

The Certificate Import wizard appears.

10. Verify that the Store location is Local Machine and click Next.
11. Click Browse, select the rootCA.crt file to import, and click Next.
12. Select the Place All Certificates in the Following Store option, select Personal, and click Next.
13. Click Finish.

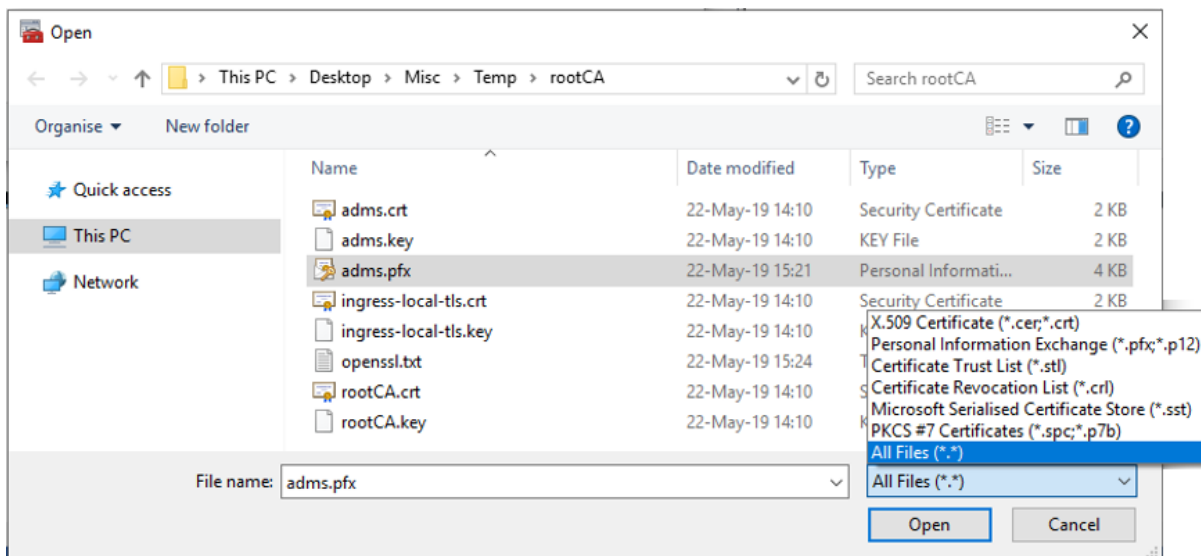
The root certificate appears in the Certificates list.



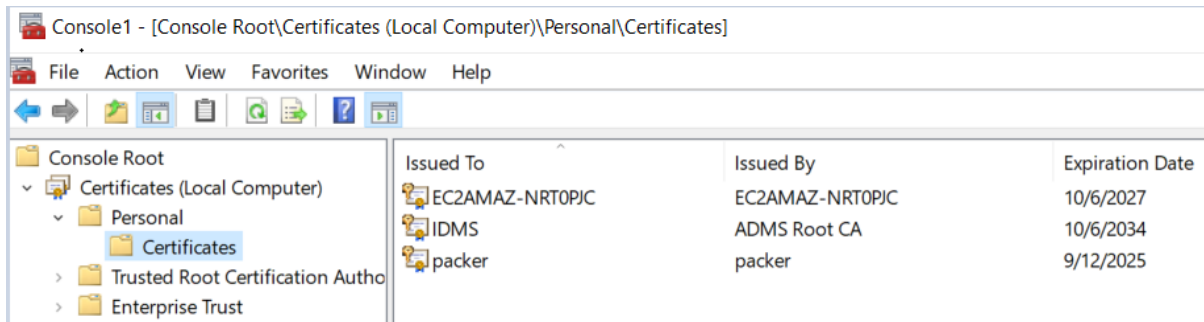
- Perform the same operations to import the ADMS PFX file with a certificate and the associated private key (adms.pfx). Enter the password that you used when generating the certificate (see [Generating Certificates](#)).

Note

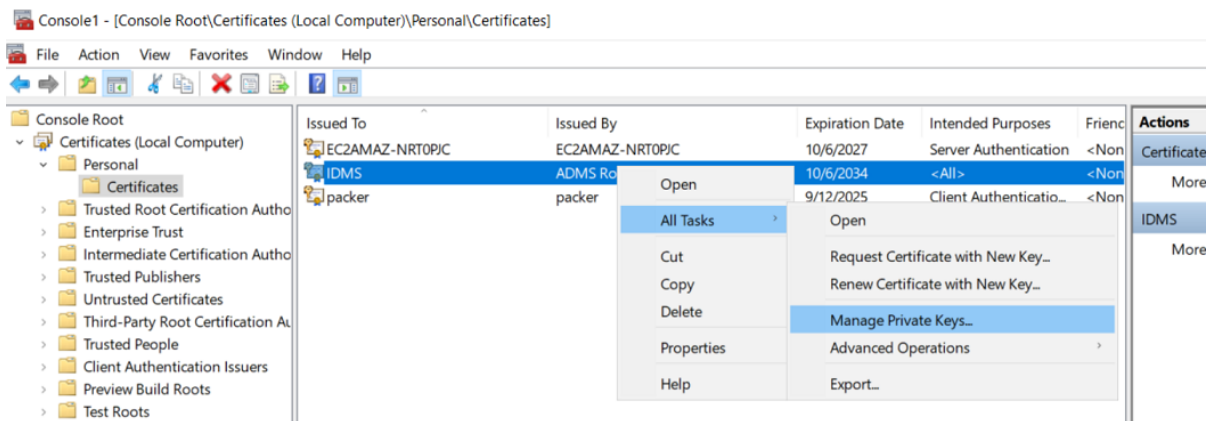
To browse for a .pfx file, select the Personal Information Exchange or All Files filter in the Open dialog box.



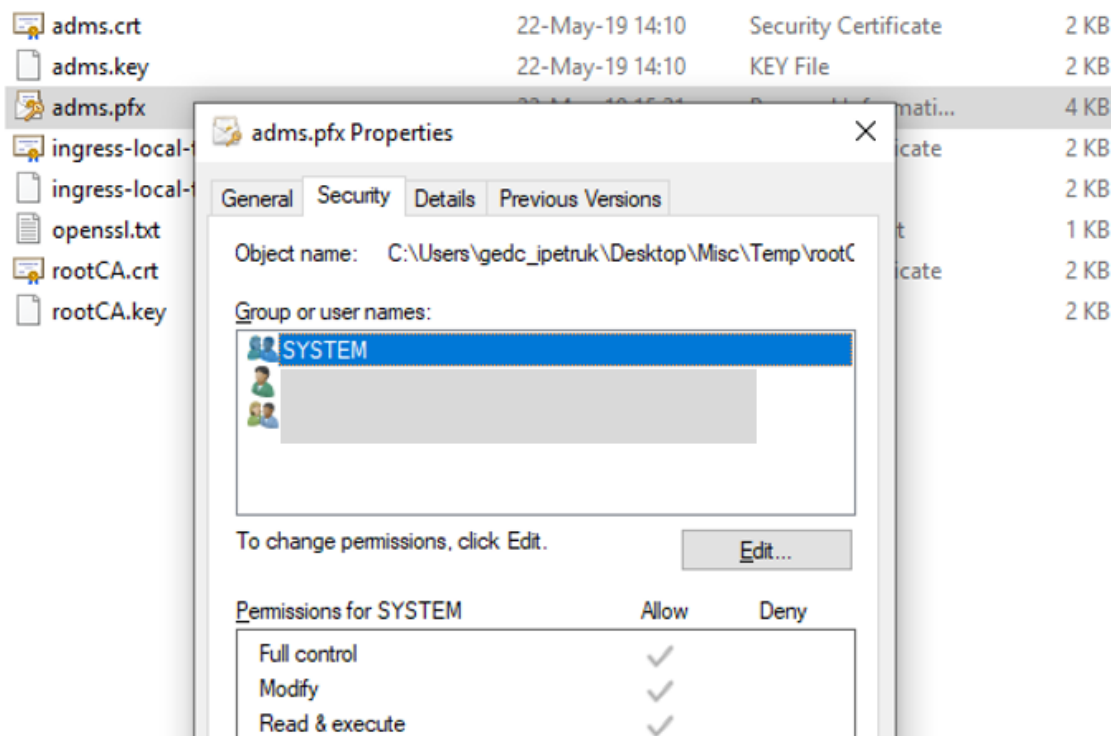
- Verify that the Root certificate authority certificate (ADMS Root CA) is listed in the Personal folder under Certificates (Local Computer).



16. Right-click the ADMS server certificate (IDMS), point to All Tasks, and click Manage Private Keys.



Alternatively, open the Security tab of the certificate Properties display.

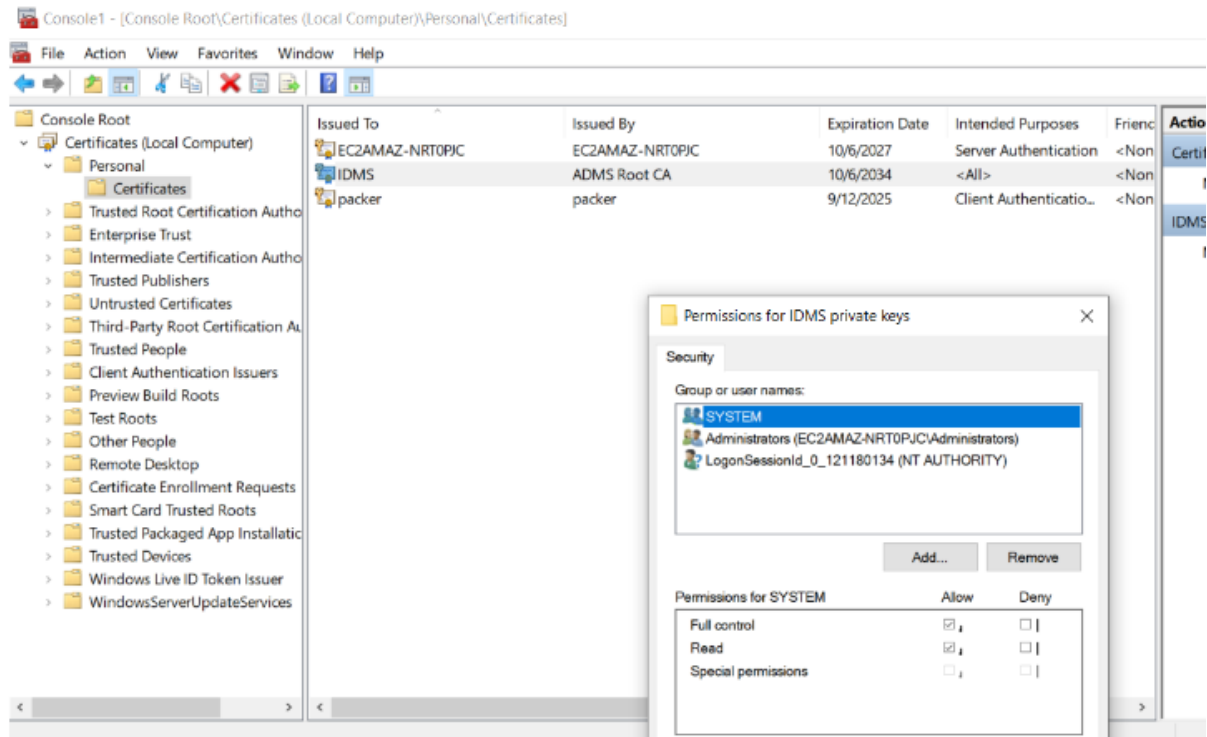


17. Add the user or service account that is used to run ADMS servers (or select Everyone, which is less secure) and give Read permissions.

To read from Trusted Root Certification Authorities, permission must be granted for users that do not have read permissions.

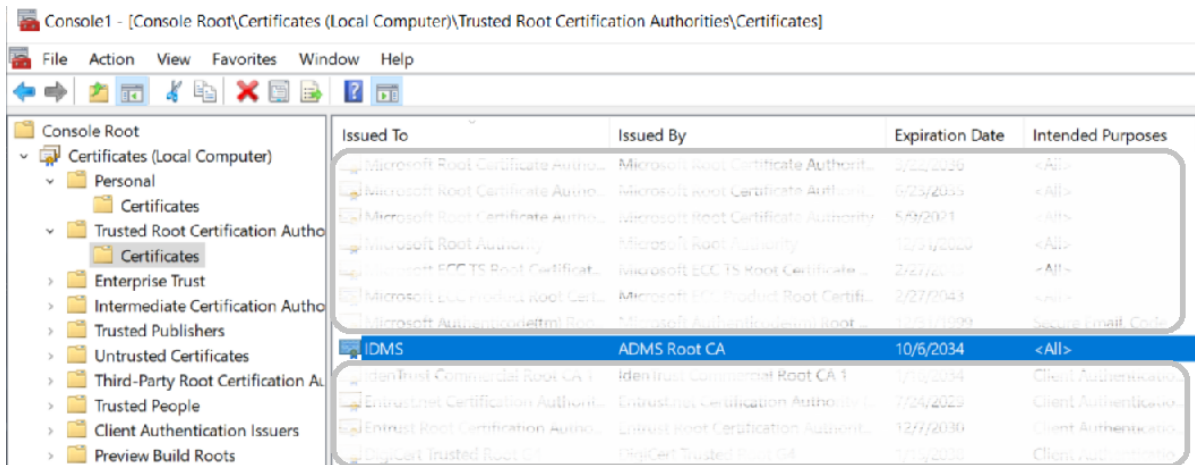
Important

Make sure you are giving permissions to the private key, not the public key.



This allows access to the private key of the imported certificate to the user (service account) that is used to run ADMS servers. Otherwise, the servers cannot serve data in HTTPS mode.

18. Copy the IDMS certificates using the context menu.
19. Navigate to the Certificates > Trusted Root Certification Authorities > Certificates folder.
20. Paste the IDMS certificates using the context menu.



21. Add or copy the IDMS certificates to the Certificates - Current User store.

This step is optional. It should be performed if you fail to bind a certificate to a port as described further.

4.7. Deploying Internal Notifications to Kubernetes

Note

It is assumed that you already have a "vanilla" Kubernetes cluster.

Note

All machines must be in one network and must be able to ping each other and exchange data requests and responses.

Note

Multi-Site Configuration Support

Internal Notifications is not site-aware and does not support multi-site configuration.

To deploy Internal Notifications:

1. Navigate to <Notifications_Deployment_Folder>.
2. Execute the following commands:

```
helm repo add adms-helm https://dig-grid-artifactory.apps.ge.com/artifactory/api/helm/adms-helm
--username ${GRIDART_USR} --password ${GRIDART_PSW}
# create image pull secret
kubectl create secret docker-registry ${GRIDART_SECRET_NAME} --docker-server=${DGA_REGISTRY}
--docker-username=${GRIDART_USR} --docker-password=${GRIDART_PSW} -n etd
kubectl patch serviceaccount default -n etd -p '{"imagePullSecrets": [{"name": "docker-registry"}]}'
```

```
# create PG secret
kubectl get secret postgres-db-pg-cluster-postgres-secret -n foundation-cluster-zerotrust -o yaml
| grep -v '^\\s*namespace:\\s' | kubectl apply -n etd -f - || true
export VALUES_ARGS="--values internal-notifications-values.yaml \
--set global.clusterExternalUrl=https://${CLUSTER_EXTERNAL_DNS} \
--set internal-notifications-
ui.callServerExternalGatewayRoute.endpoints[0]=https://${WIN_CALL_SERVER_IP}:8443"
helm upgrade -i internal-notifications adms-helm/internal-notifications -n etd --version 16.13.0
--wait --timeout=15m \\$VALUES_ARGS
```

ArgoCD Based Deployment

Note

Unlike the helm-only use case, ArgoCD deployment requires pushing the configuration manifests to a Git repository additionally. In either case, make a note of the location of each configuration manifest in Git, as they will be used to configure the internal-notifications ArgoCD application.

The command below assumes that your kubectl client is configured with the argocd cluster kubeconfig file, meaning you have access to it. If not, please contact the argocd admin user/team to obtain the argocd credentials and log in using the classical way.

```
kubectl config set-context --current --namespace=argocd
argocd login --core

helm repo add adms-helm https://dig-grid-artifactory.apps.ge.com/artifactory/api/helm/adms-helm
--username ${GRIDART_USR} --password ${GRIDART_PSW}
kubectl create secret docker-registry ${GRIDART_SECRET_NAME} --docker-server=${DGA_REGISTRY} --docker-
username=${GRID_ARTIFACTORY_CREDS_USR} --docker-password=${GRID_ARTIFACTORY_CREDS_PSW} -n
"${OA_DEPLOYMENT_NAMESPACE}" || true
kubectl patch serviceaccount default -n "${OA_DEPLOYMENT_NAMESPACE}" -p '{"imagePullSecrets":
[{"name": "docker-registry"}]}'
# create PG secret
kubectl get secret postgres-db-pg-cluster-postgres-secret -n "${PLATFORM_NAMESPACE}" -o yaml | grep -v
'^\\s*namespace:\\s' | kubectl apply -n "${OA_DEPLOYMENT_NAMESPACE}" -f - || true
kubectl apply -f internal-notifications-uaa-client.yaml
argocd repo add https://dig-grid-artifactory.apps.ge.com/artifactory/api/helm/adms-helm \
--type helm \
--name adms-helm \
--username ${GRIDART_USR} \
--password ${GRIDART_PSW} \
--upsert

argocd repo add https://github.build.ge.com/IDMS/internal-notifications-charts.git \
--username ${GIT_USR} \
--password ${GIT_PSW} \
--upsert

argocd app create -f internal-notifications.yaml --repo https://github.build.ge.com/IDMS/internal-
notifications-charts.git \
--helm-set-string
```

```
spec.sources.values.adhocGlobals.global.clusterExternalUrl=https://${CLUSTER_EXTERNAL_DNS} \
--helm-set-string internal-notifications-
ui.callServerExternalGatewayRoute.endpoints[0]=https://${WIN_SERVER}:8443 \
--helm-set global.zitadel.enabled=${APISIX_ZITADEL_ENABLE} \
--upsert
argocd app wait internal-notifications --sync --timeout 600
argocd app wait internal-notifications --operation --timeout 600
argocd app wait internal-notifications --health --timeout 600
```

To verify the Internal Notifications deployment:

1. Execute the following command:

```
helm list
```

2. Verify that Internal Notifications and Foundation Base features are deployed.
3. Execute the following command:

```
kubectl get pods --namespace etd
```

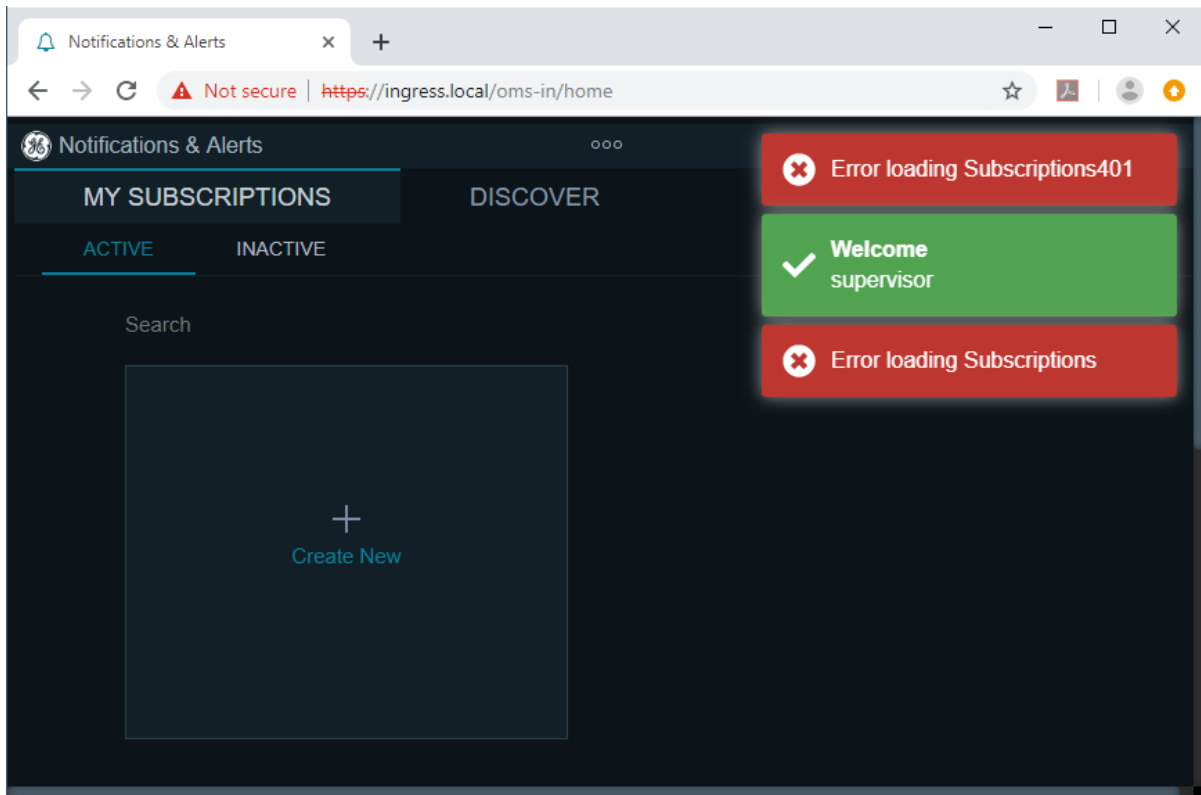
4. Verify that Internal Notifications UI, Service, and Database containers are listed. It may take several minutes for the containers to start up.

```
212787715@G2LP5353E MINGW64 /
$ kubectl get pods -n etd
```

NAME	READY	STATUS	RESTARTS	AGE
notify-internal-notifications-consumer-75ff854ff7-72gtj	2/2	Running	0	5m20s
notify-internal-notifications-service-0	2/2	Running	0	5m20s
notify-internal-notifications-ui-7b9bf68c46-4kz86	2/2	Running	0	5m20s

5. Navigate to <https://service.ingress.local/oms-in/home> in a web browser. If successful, a login page will appear.
6. Log in to the application.

The Internal Notifications interface should open. The application might not show data yet, for example, if the Core Notification Server is not running or the database is not set up.



4.8. Configuring the Core Notification Server

To configure the **CoreNotificationServerConfig.xml** file:

1. Navigate to the %IDMS_ROOT%\config\interfaces folder.
2. Edit the CoreNotificationServerConfig.xml file and configure all needed values, including the OMS Interface Server URL, UAA endpoint, and journal file paths.

For example:

```
<?xml version="1.0" encoding="utf-8"?>
<CoreNotificationServerConfig>
<OmsInterfaceServerURL>http://localhost:18802/api/adms/oms/v2/</OmsInterfaceServerURL>
<ConsumerServerURL>http://service.<Kubernetes_Cluster_URL></ConsumerServerURL>
<UaaEndPoint><Url>https://zitadel.<Kubernetes_Cluster_URL>/oauth/v2/token</Url></UaaEndPoint>
<LogFileName>%IDMS_ROOT%\log\CoreNotification\CoreNotificationServer.txt</LogFileName>
<MaxMessagesInLogFile>10000</MaxMessagesInLogFile>
<EnableDebugLog>true</EnableDebugLog>
<OmsDBConfig>
<EnableJournalLogging>false</EnableJournalLogging>
<OmsCustomerFile>%IDMS_ROOT%\models\customer\FantasyIslandCustomers.bpcl</OmsCustomerFile>
<OmsCallJournalFilePath>%IDMS_ROOT%\dynamics\server\OMSServer\CallJournal\</OmsCallJournalFilePath>
>
<OmsArchiveJournalFilePath>%IDMS_ROOT%\dynamics\server\OMSServer\ArchiveJournal\</OmsArchiveJournalFilePath>
</CoreNotificationServerConfig>
```

```
<OmsIncidentJournalFilePath>%IDMS_ROOT%\dynamics\server\OMSServer\IncidentJournal\</OmsIncidentJournalFilePath>
<OmsCrewJournalFilePath>%IDMS_ROOT%\dynamics\server\OMSServer\CrewJournal\</OmsCrewJournalFilePath>
<OmsOrderJournalFilePath>%IDMS_ROOT%\dynamics\server\OMSServer\OrderJournal\</OmsOrderJournalFilePath>
<CodedTranslationConfig>%IDMS_ROOT%\config\shared\CodedTranslationConfig.xml</CodedTranslationConfig>
<OmsSnapshotFilePath>%IDMS_ROOT%\dynamics\server\OMSServer\Snapshot\</OmsSnapshotFilePath>
<OmsCrewEmployeeFile>%IDMS_ROOT%\models\crew\FantasyIslandCrewModel.Crew</OmsCrewEmployeeFile>
</OmsDBConfig>
</CoreNotificationServerConfig>
```

3. Save the file.
4. Restart the Core Notification Server if it was already running.

4.8.1. Configuring Credentials to Access the Kubernetes Cluster

Core Notification Server needs to connect to the Core Notification Consumer, which runs inside the Kubernetes cluster. For the Core Notification Server to access the Kubernetes cluster, add environment variables that define service connection credentials:

1. Open a command prompt.
2. Execute the following commands to add environment variables for the applications.

```
SETX CORENOTIFICATIONSERVER_UAA_AUTH_CLIENTID=internal-notification-app-admin
SETX CORENOTIFICATIONSERVER_UAA_AUTH_CLIENTSECRET=internal-notification-app-admin-secret

# Get the Zitadel Project ID
kubectl get configmap -n foundation-cluster-zerotrust zitadel-projects -o
jsonpath='{.data.foundation\.foundation\.id}'

# Get CORENOTIFICATION UAA Client ID and Secret
kubectl get secret internal-notification-app-admin -n etd -o jsonpath='{.data.clientId}' | base64
--decode
kubectl get secret internal-notification-app-admin -n etd -o jsonpath='{.data.clientSecret}' |
base64 --decode
```

Note

Core Notification Server uses OMS Interface Server names in environment variables.

4.9. Configuring the Call Server

To configure the Call Server:

1. Edit the %IDMS_ROOT%\config\server\CallServerConfig.xml file.
2. Make sure that the EnableCORS parameter is set to True:

```
<EnableCORS>true</EnableCORS>
```

3. Make sure that the Call Server plugin host and port values correspond to the callServerExternalGatewayRoute values in the the manifest.yaml file. For example:

```
<UriScheme>https</UriScheme>  
<UriHost>+</UriHost>  
<UriPort>8443</UriPort>  
<UriPath>api</UriPath>
```


5. Running Internal Notifications

1. Make sure that the Internal Notifications pods in Kubernetes are running.
2. Make sure that the ADMS system is running, including the OMS Server and OMS Interface Server.
3. From the Windows Start menu, navigate to All Programs > Eterra > Distribution > ADMSInterfaces > Core Notification Server.

The Internal Notifications application is now ready to use. In a web browser, open <https://service.ingress.local/oms-in/home> and manage user subscriptions.

- The My Subscriptions tab shows your current subscriptions.

MY SUBSCRIPTIONS DISCOVER ADMINISTRATOR

ACTIVE INACTIVE

← Back
New Subscription

Details

NAME
Subscription

TYPE
Affected Customers

Notification Criteria

☒ Notify on Incident Working Status change
☒ Notify on ETR change

Search Customer ☐ Business

First Name john

Last Name

Account # Search Clear

SELECT

Customer Name	Account Number	
Pratt John	555211120	+
Root John	555110941	+
Rudoff John	555110963	+
Rowland John	555360193	+
Kellias John	555372695	+
Benton John	555038632	+
Schormann John	555145891	+

Method

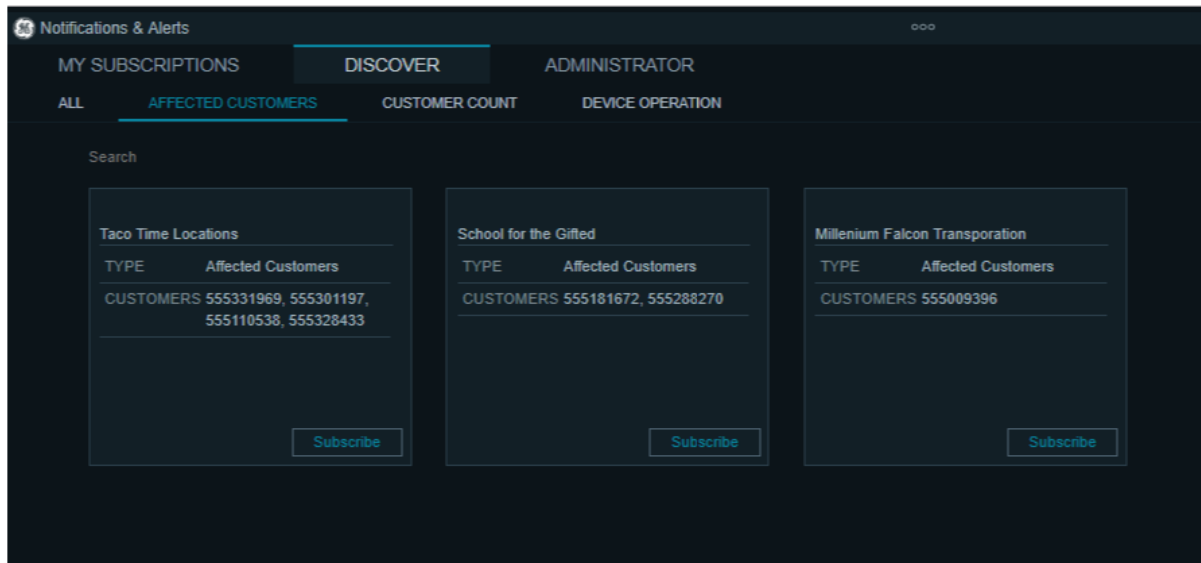
EMAIL is required PHONE NUMBER is required

EMAIL PHONE NUMBER

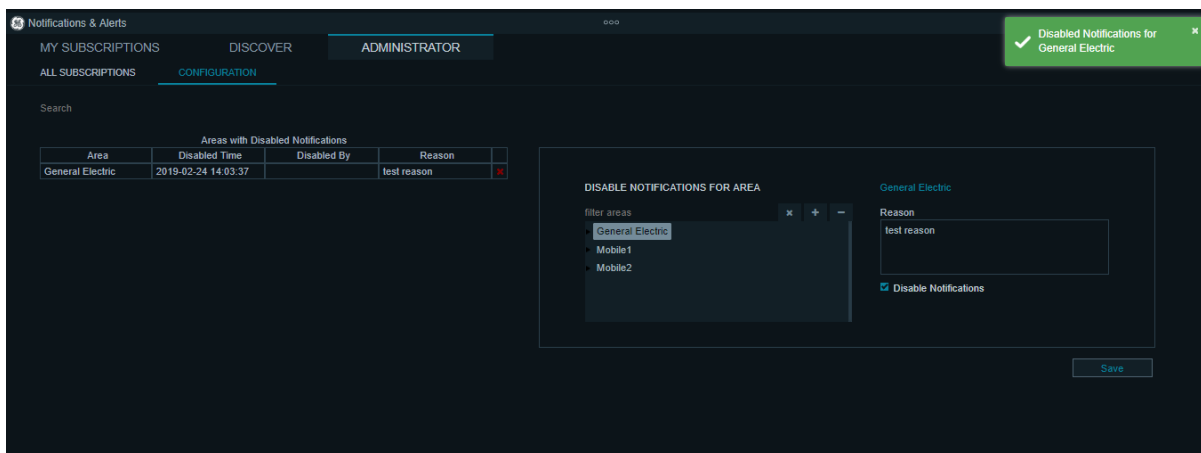
CUSTOMERS

Customer Name	Account Number	
Root John	555110941	✗
Rudoff John	555110963	✗
Rowland John	555360193	✗

- The Discover tab shows subscriptions for other users in the system. Use the Subscribe button to copy a subscription for yourself.



- The Administrator tab allows you to manage all subscriptions in the system. This tab is only shown for users with administrator privileges.



6. Maintenance and Troubleshooting

6.1. Verifying the Setup

To verify that you have correctly configured and run the system:

1. Create a subscription in the Internal Notifications application.
2. In ADMS, create an incident that is associated with the created subscription. For example, if you have a Device Operation subscription for breakers, create an incident on a breaker.
3. Open a new command prompt as an administrator.
4. Get the name of the Kubernetes pod where the internal notifications and foundation system is installed:

```
kubectl get pods --namespace etd
kubectl get pods --namespace foundation-cluster-zerotrust
```

5. Use the pod name to open a shell into the database container:

```
kubectl exec -it <database container name> -n <database container namespace> -c <database container name in input manifest> sh
```

For example:

```
kubectl exec -i -t -n foundation-cluster-zerotrust postgres-db-pg-cluster-0 -c postgres -- sh -c "clear; (bash || ash || sh)"
```

6. Navigate to the folder with the database.

```
> cd Database
```

7. Enter the database:

```
psql -U apiuser -d notifica
```

8. Run the following command to verify that the new incident has been recorded in the OUTAGE table in the notification database:

```
> select * from outage;
```

```

root@kube-master:~/bsfdeploy_client

[root@kube-master bsfdeploy_client]# kubectl exec -it notify-internal-notifications-database-0 -n etd -o internal-notifications-database sh
$ psql -U postgres -d notifyA
psql (10.10 (Debian 10.10-1.pgdg90+1))
Type "help" for help.

notifyA=# select * from outage;
      incident_id      | incident_name | incident_start | affected_cust_count | customers |
-----+-----+-----+-----+-----+
be5e869d-485d-4713-847f-30270e367411 | 12000000001 | 2018-05-16 14:00:00 | 1000 | {3210,555288270} | 3F6A7B2F-B7E
| CST | South Delta | Widgeon | Locke | Oregon | General Electric |
b4c4eb18-bd2f-4880-aca8-b74171a69602 | 12000000001 | 2018-10-12 04:30:00 | 100 | {345,555301197,678,876,890} | 114913349
| CST | South Delta | Widgeon | Locke | Oregon | General Electric |
ae5e869d-485d-4713-847f-30270e367499 | 12000000007 | | | | |
| CST | West Central | Levitt | Gordon | WA | General Electric |
ff9e869d-485d-4713-847f-30270e3674ef | 12000000023 | | | | |
| NEAD | Northeast Delta | Elizabeth | Tidus | WA | General Electric |
(4 rows)

```

9. Verify that the new event has been added to the OUTAGE_EVENT table:

```
> select * from outage_event;
```

10. Verify that the INCIDENT_ID value for the incident is equal in the OUTAGE and OUTAGE_EVENT tables.

11. Verify that a new item has been added to the NOTIFICATION table and contains the correct EVENT_ID:

```
> select * from notification;
```

12. Verify that the SMTP server works correctly by checking that the "NOTIFY_TIME" field in the NOTIFICATION table is populated with the date and time when the notification was sent.

6.2. Upgrading to a Newer Version

To upgrade to a newer version of Internal Notifications using Helm, enter the following:

```
helm upgrade --namespace etd --name notify <database container namespace>
```

6.3. Enable Debug Logging

To enable debug logging in internal notification:

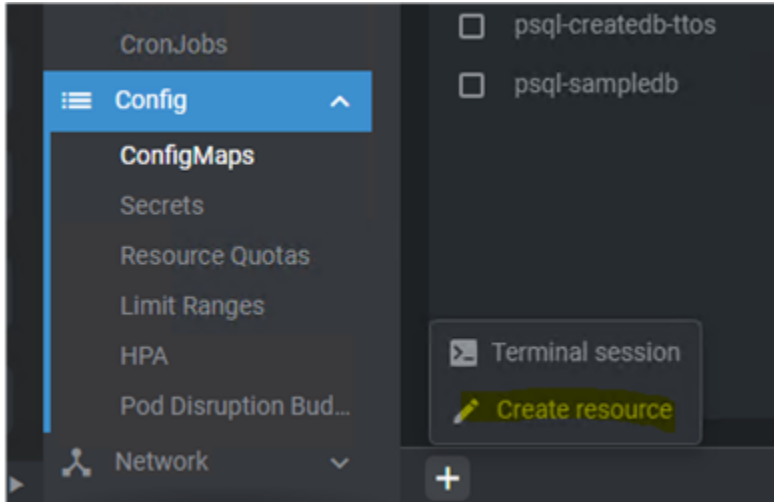
1. Kubectl strategy
 - a. Create a file named configmap-appsettings.yaml.
 - b. Add contents from [Appsettings.json](#) into the file.
 - c. Run `kubectl apply -f configmap-appsettings.yaml`.
 - d. Edit the StatefulSet

```
kubectl edit statefulset notify-internal-notifications-service -n etd
```

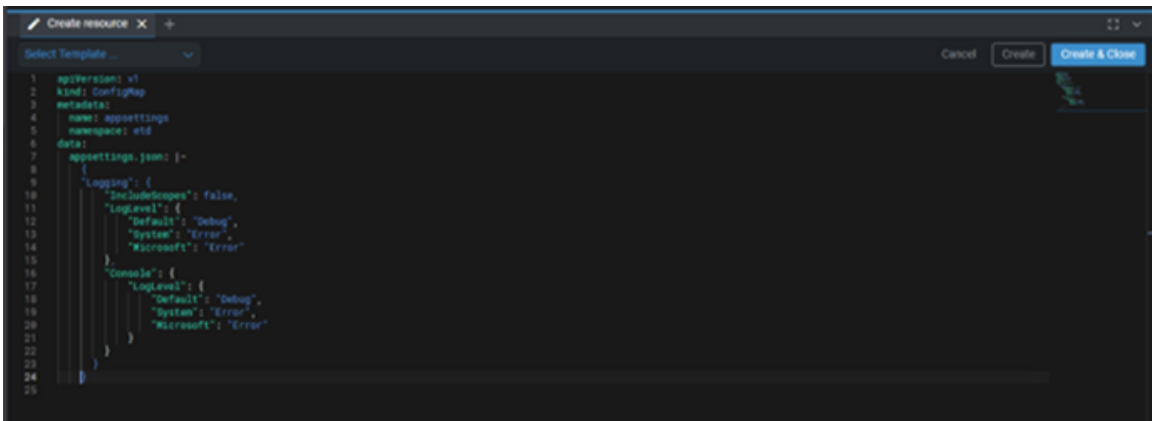
- e. Add the highlighted lines from the [Statefulset config](#). Modify the volumeMounts and volumes sections.

2. Kubernetes Lens strategy

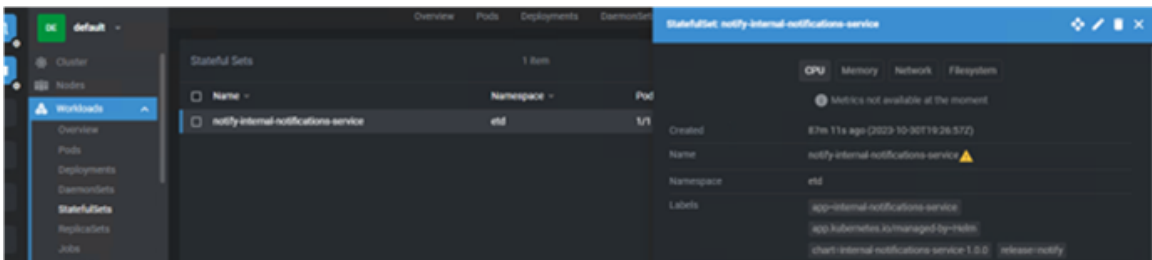
- Create resource.



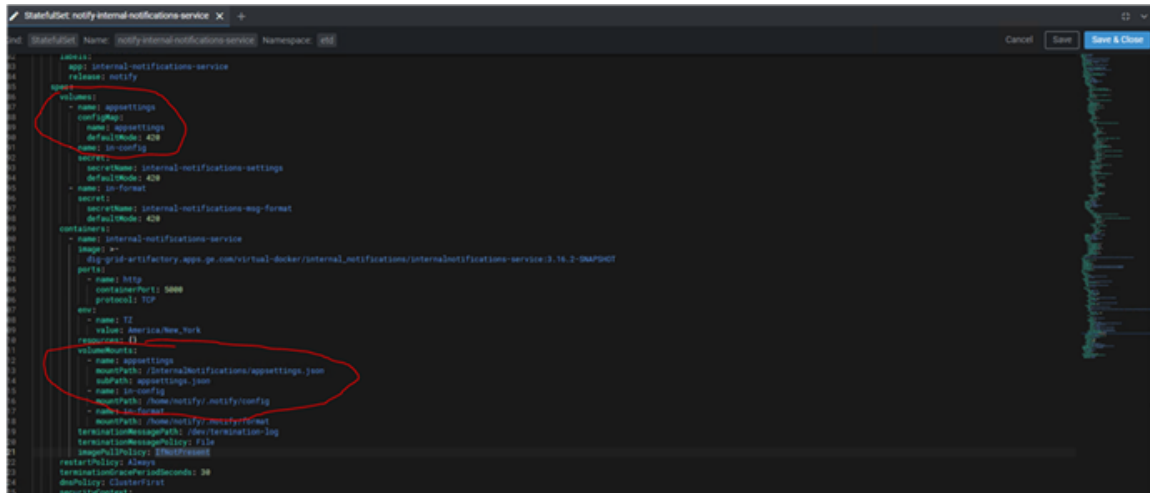
- Copy the content of the following [Appsettings.json](#). Click Create and Close.



- Edit the StatefulSet in the etd Namespace.



- Modify the volumes and volumeMounts options of [Statefulset config](#) file, and then save the changes.



6.3.1. Appsettings.json

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: appsettings
  namespace: etd
data:
  appsettings.json: |-
    {
      "Logging": {
        "IncludeScopes": false,
        "LogLevel": {
          "Default": "Debug",
          "System": "Error",
          "Microsoft": "Error"
        },
        "Console": {
          "LogLevel": {
            "Default": "Debug",
            "System": "Error",
            "Microsoft": "Error"
          }
        }
      }
    }
  }
```

6.3.2. Statefulset config

```
volumeMounts:
  - mountPath: /InternalNotifications/appsettings.json
    name: appsettings
    subPath: appsettings.json
```

```

- mountPath: /home/notify/.notify/config
  name: in-config
- mountPath: /home/notify/.notify/format
  name: in-format

...

volumes:
- configMap:
    name: appsettings
  name: appsettings
- name: in-config
  secret:
    secretName: internal-notifications-settings
- name: in-format
  secret:
    secretName: internal-notifications-msg-format

```

6.4. Enabling Additional Logging

The default logging level for the internal-notifications-service container (Default log) is "Info". To change the logging level to "Debug", add the following environment variable entries in the manifest file when deploying the application:

```

Logging:LogLevel:Default=Debug
Logging:Console:LogLevel:Default=Debug

```

You can also enable additional logging for the System and Microsoft libraries (these libraries have the default logging level set to "Error", because logs can be very verbose). To change the logging level to "Info", add the following environment variable entries in the manifest file when deploying the application:

```

Logging:LogLevel:System=Information
Logging:Console:LogLevel:System=Information
Logging:LogLevel:Microsoft=Information
Logging:Console:LogLevel:Microsoft=Information

```

You can also enable logging in the PostgreSQL database container (Database log) using the `log_statement` and `client_min_messages` parameters in `postgresql.conf`. The database runs in a separate container and has its own console and logs. Therefore, the database log is separate from the internal-notification-service log.

6.5. Configuring Automatic Database Clean-Up

You can configure the database to automatically delete records older than a specified duration when the number of records exceeds a specified number. This prevents database tables from

getting too big and therefore impacting performance.

Note

This feature is disabled by default.

You can configure this feature by modifying the `purge_parameters` table in PostgreSQL.

```
notifyA=# select * from purge_parameters;
 type | step | count | auto_purge 
-----+-----+-----+-----
 1 week | day  | 20000 | f
(1 row)
```

Figure 7. Default Values in `purge_parameters` Table

The following fields are included in the `purge_parameters` table:

- **type:** The duration of records to be retained when the count threshold is exceeded.
For example, a value of "1 week" would guarantee that all incidents that have started within the last week would NOT be deleted.
- **step:** This variable is used in the `date_trunc` function and is related to the precision in the duration calculation. It is recommended to leave the value at the default of "day".
- **count:** This is the minimum number of incidents that needs to be reached before the `purge_notification` function is triggered.
For example, if this value is set at 20000, the outage table would not have any information deleted until the 20001st record is added.
- **auto_purge:** This enables ("true") or disables ("false") the `purge_notification` feature. The default value is "false".

6.6. Deleting Records in the Outage Table

In a non-production environment for testing, before you create any new subscriptions you should delete existing records in the outage table in the Internal Notification PostgreSQL database. This is because a non-production environment will likely contain many records of incidents that were deleted from the thick client via clearing dynamics, but *not* deleted from the Internal Notifications database. Therefore, the records should be deleted in the IN database to avoid a scenario where potentially thousands of notifications are sent. This would bog down the SMTP and the volume of emails could cause an Outlook error for the user.

To delete incident information from the outage table:

1. Log into the database and execute the following command:

```
TRUNCATE outage CASCADE;
```


2. Verify that the "outage", "outage_event", and "notification" tables are empty.

6.7. Troubleshooting

6.7.1. Notifications Contain Duplicate Service Center Data

Symptom: Service Center information is duplicated in received notifications.

Cause: In the organizational hierarchy mode, level 1 is equal to Service Center.

Solution: Remove the outageServiceCenter entry or remove zone1 entries from the `msgFormat.json` file.

6.7.2. Area Filter is Blank

Symptom: Area field is blank when creating or updating a notification subscription.

Cause: The connection between the Core Notification Server and the OMS Interface Server is not established.

Solution: Ensure that the OMS Interface Server is up and running. Check that the `OmsInterfaceServerURL` value in the `CoreNotificationServerConfig.xml` file points to the OMS Interface Server web service URL.

6.7.3. Customer Search is not Working

Symptom: Customer search list is blank when creating or updating a notification subscription.

Cause: The connection between the Core Notification Server and the Call Server is not established.

Solution: Ensure that the Call Server is up and running. Check that the `customerSearchService` value in the input manifest file points to the Call Server host and port. Check that Call Server is running in HTTP mode.

Note

HTTPS is not supported in this release.

6.7.4. Subscribers do not Receive Email Notifications

Symptom: Subscribers do not receive email notifications.

Cause 1: SMTP server fails to send a message. If there was an actual attempt to send a notification via the SMTP server, a log message in the internal-notifications-service container (Default log) at the

Info level shows that the notification was successfully sent or a log message at the Warn level shows that the SMTP server attempted and failed to send a notification. The Debug level in the SMTP sending code gives an additional message "about to send an SMTP message". If there is an actual error in the sending process, the System and Microsoft logs can provide more information.

Cause 2: There is a subscription configuration issue or modeling issue. If there is a subscription configuration issue or modeling issue, the database trigger does not raise an indication that an SMTP message must be sent. If the database trigger (written in PL/pgSQL) does not match an event (incident) with a subscription, the system never attempts to send a notification. (In this case, SMTP sending code logs do not provide information, because the sending function only runs if the database trigger fires.)

Solution:

1. Enable additional logging, as described in [Enabling Additional Logging](#).
2. Check the Default, System, Microsoft, and Database logs.
3. Fix the SMTP server, subscription, or model, as appropriate.